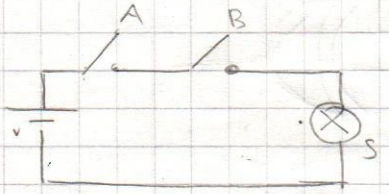
HOJA

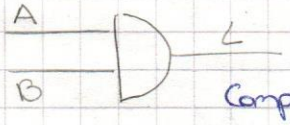
FECHA T D = T

Operadores logicos

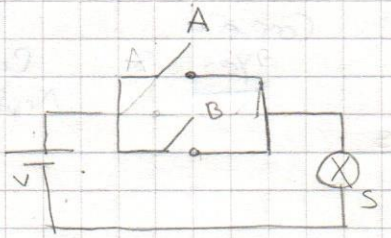


A	B	S
0	0	0
0	1	0
1	0	0
1	1	1

$S = A \cdot B$



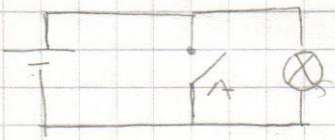
Compuerta "Y" - "and"



A	B	S
0	0	0
0	1	1
1	0	1
1	1	1

$S = A + B$

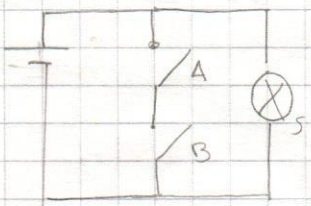
Compuerta "O" - "or"



A	S
0	1
1	0

$S = \overline{A}$

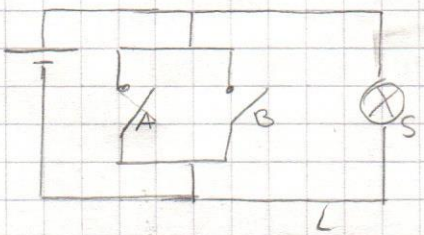
Compuerta "Not" - inversor



A	B	S
0	0	1
0	1	1
1	0	1
1	1	0

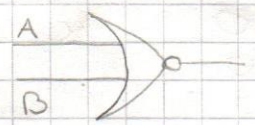
$S = \overline{A \cdot B}$

Compuerta "nand" - and negada



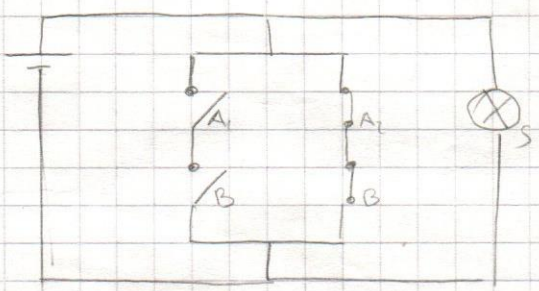
A	B	S
0	0	1
0	1	0
1	0	0
1	1	0

$S = \overline{A + B}$



Compuerta "Nor" - No o

Si se doc a_1 se cierra a_2

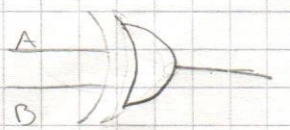


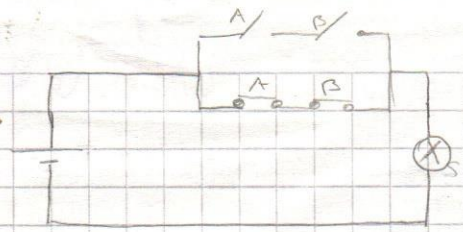
A	B	S
0	0	0
0	1	1
1	0	1
1	1	0

$S = A\overline{B} + \overline{A}B$

$S = A \oplus B$

Compuerta "Xor" - or exclusiva



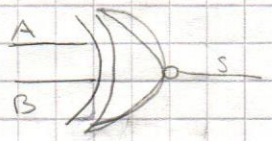


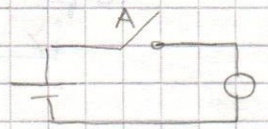
A	B	S
0	0	1
0	1	0
1	0	0
1	1	1

$S = \bar{A}.\bar{B} + A.B$

$S = A \odot B$

Compuerta XNOr - nor exclusiva





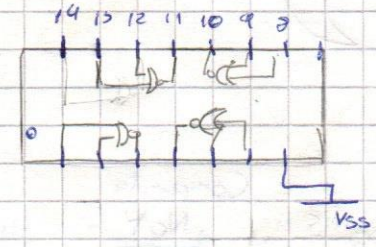
A	S
0	0
1	1

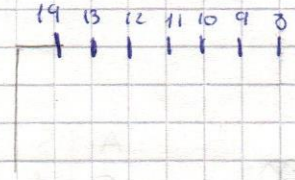
$S = A$

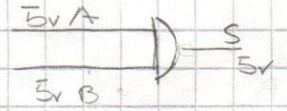
Compuerta Yes. Si

Baffer
Amplificador de corriente

4001







Si entra 4v igual sale 5v

. Compuerta de mas de 2.

A	B	C	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

NOTA

Algebra de boole

HOJA N°

FECHA

- Principio de dualidad
  $A+(B.C)=1$
  $A.(B+C)=0$

- Ley de morgan
  $\overline{A+B} = \bar{A}.\bar{B}$
  $\bar{A}+\bar{B} = \overline{A.B}$

- Ley de absorción
  $A+(A.B)=A$
  $A.(A+B)=A$

- Ley de
  $A=\bar{\bar{A}}$
  $A+A=A$

- Elemento neutro
  $A+1=1$
  $A.0=0$
  $A+0=A$
  $A.1=A$
  $A.\bar{A}=0$
  $A+\bar{A}=1$
  $\bar{A}+AB+\bar{B}=1$

$A+(\bar{A}B) = A+B$

- Expancion a maxiterminos

$A = A(B+\bar{B}) = AB+A\bar{B}$

se mult. a cada termino que falte completar

- Expandir a minter (suma de prod)

$A = A+(B\bar{B}) = (A+B)(A+\bar{B})$

Se suma a cada termn. q falte completar

- Expresiones canonicas

min → $\bar{A}\bar{B}+A\bar{B} \rightarrow \Sigma m(0,2)$

max → $(A+B)(A+\bar{B}) : \Pi M(1,0)$

A	B	S	min	max
0	0	1	m_0	M_3
0	1	0	m_1	M_2
1	0	1	m_2	M_1
1	1	0	m_3	M_0

NOTA

NOTA

Unidad 4

HOJA N° 1

FECHA

Sistema de numeración

Decimal

$542_{(10)} = 5 \cdot 10^2 + 4 \cdot 10^1 + 2 \cdot 10^0$

$= 500 + 40 + 2$

Binario

bin a dec $10010_{(2)} = 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 0 + 0 + 4 + 0 + 1 = 37_{(10)}$

dec a bin

40 \| 2
0 20 \| 2
0 10 \| 2
0 5 \| 2
1 2 \| 2
0 1

$40_{(10)} = 101000_{(2)}$

dec a octal x \| 8

dec a hexa x \| 16

Octal (0-7)

Octal a bin $27_{(8)} = (\text{dos en bin})(7 \text{ en bin}) = \overline{010\,111}_{(2)}$

bin a octal $\overline{1\,101\,011\,101}_{(2)} = \overline{001\,101\,011\,101}_2 = 1535_8$

Separo en 3 partes a partir de la derecha y si queda colgado uno, le agrego ceros

Hexadecimal (0-F)

bin a hexa $\overline{1\,0011\,1011}_2 = 0001\,0011\,1011_2 = 13B_{16}$ $\quad \overline{1011,1010\,1}_2 = B,A8_{16}$

hexa a bin $8FA_{16} = 1000\ 1111\ 1010_2$

$AF,5_{16} = 1010\ 1111,0101_2$

	Hexa	octal
0	0 0 0 0	0 0 0
1	0 0 0 1	0 0 1
2	0 0 1 0	0 1 0
3	0 0 1 1	0 1 1
4	0 1 0 0	1 0 0
5	0 1 0 1	1 0 1
6	0 1 1 0	1 1 0
7	0 1 1 1	1 1 1
8	1 0 0 0	
9	1 0 0 1	
A	1 0 1 0	
B	1 0 1 1	
C	1 1 0 0	
D	1 1 0 1	
E	1 1 1 0	
F	1 1 1 1	

NOTA

MSB = más sig
LSB = menos sig

Codificación son formas de relacionar 2 elementos

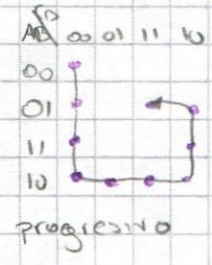
Tipos de codigos

▲ Ponderados: Están formados por un polinomio y xq se le asigna un peso binario a cada variable

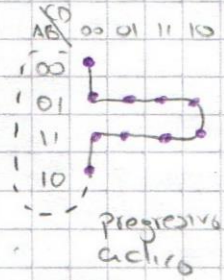
Digital Decmal	BCD (Natural) $2^3\ 2^2\ 2^1\ 2^0$ A B C D	AIKEN 2 4 2 1 A B C D	5 4 2 1
0	0 0 0 0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 0	0 0 1 0
3	0 0 1 1	0 0 1 1	0 0 1 1
4	0 1 0 0	0 1 0 0	0 1 0 0
5	0 1 0 1	1 0 1 1	1 0 0 0
6	0 1 1 0	1 1 0 0	1 0 0 1
7	0 1 1 1	1 1 0 1	1 0 1 0
8	1 0 0 0	1 1 1 0	1 0 1 1
9	1 0 0 1	1 1 1 1	1 1 0 0

▲ libres: no están formados por polinomios sino por un algoritmo no aritmético (llamados progresivos cuando pasan de una combinación a la sig. cambia un dígito)

Decimal	Exceso 3	Progresivo	Progresivo Cíclico	Johnson	Gray
0	0 0 1 1	0 0 0 0	0 0 0 0	0 0 0 0 0	0 0 0 0
1	0 1 0 0	0 1 0 0	0 1 0 0	0 0 0 0 1	0 0 0 1
2	0 1 0 1	1 1 0 0	0 1 0 1	0 0 0 1 1	0 0 1 1
3	0 1 1 0	1 0 0 0	0 1 1 1	0 0 1 1 1	0 0 1 0
4	0 1 1 1	1 0 0 1	0 1 1 0	0 1 1 1 1	0 1 1 0
5	1 0 0 0	1 0 1 1	1 1 1 0	1 1 1 1 1	0 1 1 1
6	1 0 0 1	1 0 1 0	1 1 1 1	1 1 1 1 0	0 1 0 1
7	1 0 1 0	1 1 1 0	1 1 0 1	1 1 1 0 0	0 1 0 0
8	1 0 1 1	0 1 1 0	1 1 0 0	1 1 0 0 0	1 1 0 0
9	1 1 0 0	0 1 1 1	1 0 0 0	1 0 0 0 0	1 1 0 1
					1 1 1 1
					1 1 1 0
					1 0 1 0
					1 0 1 1
					1 0 0 1
					1 0 0 0

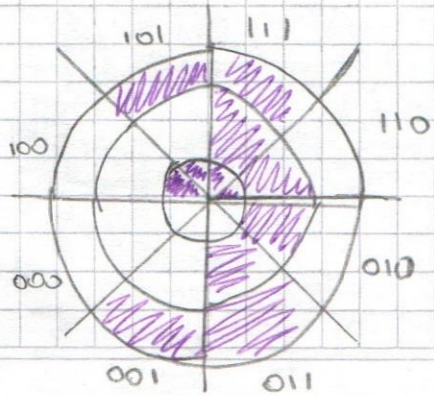


progresivo



Progresivo cíclico

Conversión de codigos: hacemos las 2 tablas de verdad, tomo las variables de la 2da como función y aplico Karnaugh



Gray

NOTA

Detector de errores

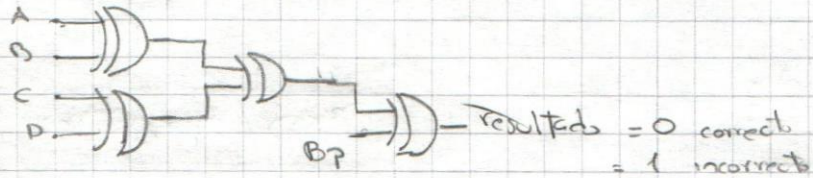
Bit de paridad (detector)

B C D	P_P (Par)	P_I (Impar)
0 0 0	0	1
0 0 0 1	1	0
0 0 1 0	1	0
0 0 1 1	0	1
0 1 0 0	1	0
0 1 0 1	0	1
0 1 1 0	0	1
0 1 1 1	1	0
1 0 0 0	1	0
1 0 0 1	0	1

$P_P = A \oplus B \oplus C \oplus D$

$P_I = \overline{A \oplus B \oplus C \oplus D}$

P/recibirlo



Hamming (detector y corrector)

$k_1 = m_3 \oplus m_5 \oplus m_7 \oplus m_9$

$n = 4$ $\quad 2^k \geq k + n + 1$ $\quad k$ = bit de paridad, n = datos

$\therefore 2^3 = 3 + 4 + 1 \quad \therefore k = 3$

- Ejemplo: Dato = 1011 $\quad n = 4 \quad \therefore 2^3 = 3 + 4 + 1 \quad k = 3$

cuento los "1" si es par $k = 0$

Palabra a transmitir:

m_7	m_6	m_5	k_4	m_3	k_2	k_1
1	0	1	0	1	0	1

$k_1 = m_3^{=1} \oplus m_5^{=1} \oplus m_7^{=1} = 1$

$k_2 = m_3^{=1} \oplus m_6^{=0} \oplus m_7^{=1} = 0$

$k_4 = m_5^{=1} \oplus m_6^{=0} \oplus m_7^{=1} = 0$

Si se produce un error

m_7	m_6	m_5	k_4	m_3	k_2	k_1
0	0	1	0	1	0	1

error ↑

$G_1 = k_1^{=1} \oplus m_3^{=1} \oplus m_5^{=1} \oplus m_7^{=0} = 1$

$G_2 = k_2^{=0} \oplus m_3^{=1} \oplus m_6^{=0} \oplus m_7^{=0} = 1$

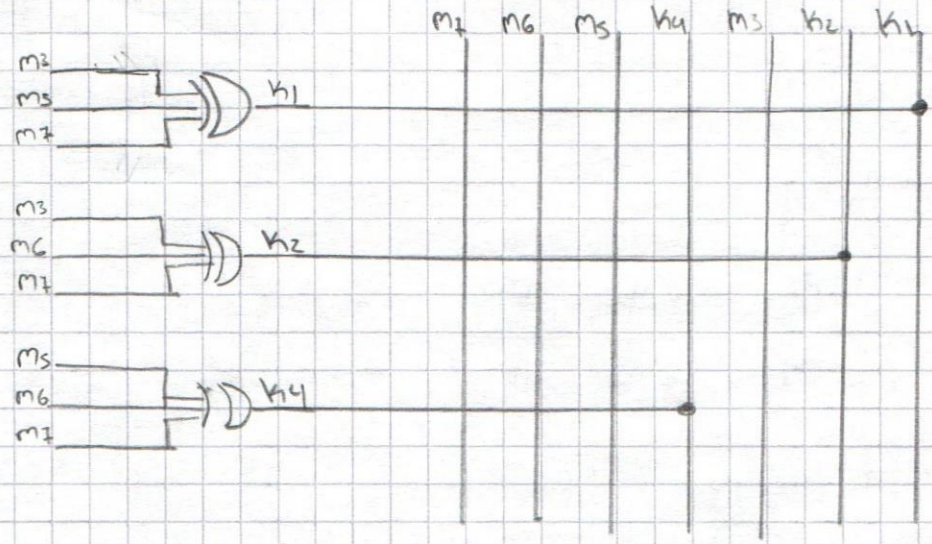
$\therefore 111_2 = 7_{10}$ error en el bit 7

$G_4 = k_4^{=0} \oplus m_5^{=1} \oplus m_6^{=0} \oplus m_7^{=0} = 1$

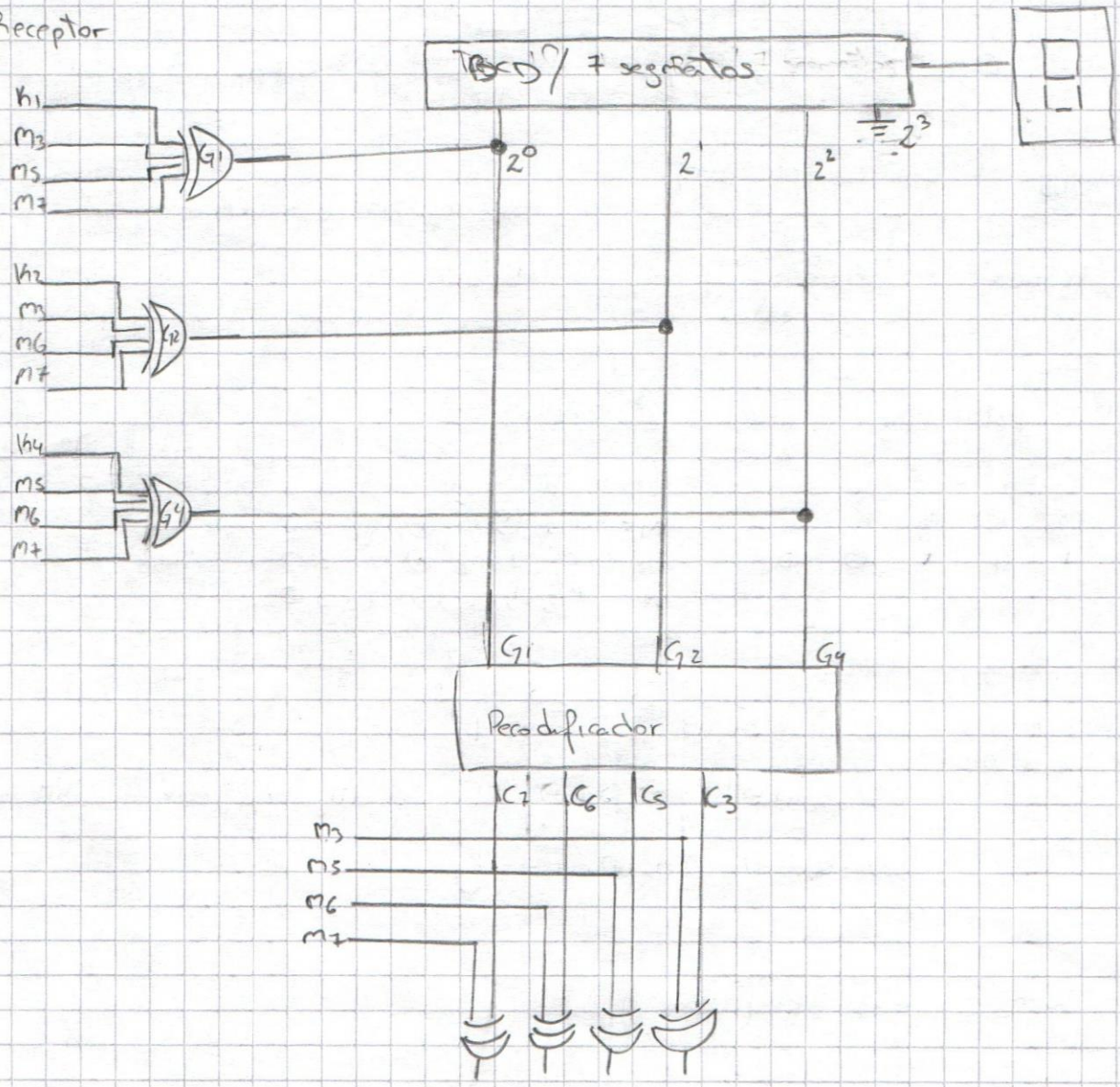
Falta método matricial (resumen)

NOTA

Transmisor



Receptor



NOTA

Hamming

Ej: 0 1 0 1 0 1 1 — 7 bits

$$2^k \geq \overbrace{n+k+1}^{T}$$

$k = 4$

$T = 11$

$2^6 \geq 7+4+1$

$64 \geq 12$

$k_1 = 2^0$

$k_2 = 2^1$

$k_4 = 2^2$

$k_8 = 2^3$

0 1 0 1 0 1 0 1 1 1

b_7	b_6	b_5	k_8	b_4	b_3	b_2	k_4	b_1	k_2	k_1	
0	1	0	1	0	0	0	0	0	0	0	$G_8 = k_8 \oplus b_5 \oplus b_6 \oplus b_7 = 1 = k_8$
0	0	0	0	1	0	1	0	0	0	0	$G_4 = k_4 \oplus b_2 \oplus b_3 \oplus b_4 \oplus = 0 = k_4$
0	1	0	0	1	0	0	1	1	0	0	$G_2 = k_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 = k_2$
0	0	0	0	1	0	1	0	1	0	1	$G_1 = k_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 1 = k_1$

es una tabla invertida

Otro metodo
- marco los k_n
- donde tengo 0 hago cero toda la columna
- Cuento cantidad de 1, si es par = 0
  Sin tener los "1" de los k_n impar = 1

$T = 01011010111$

le cubro los 0,1 de acuerdo su subindice

Matricial

$$\begin{bmatrix} b_{11} & b_{10} & b_9 & b_7 & b_6 & b_5 & b_3 & k_8 & k_4 & k_2 & k_1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 1 \\ k_8 \\ k_4 \\ k_2 \\ k_1 \end{bmatrix}$$

MSB

LSM

$k_8 = 1$

$k_4 = 0$

$k_2 = 1$

$k_1 = 1$

multiplico fila y columna después XOR

0 1 1 0 0 0 0 1 0 1 0 0 1 0 1 0 0 1

NOTA

T = 0 1 0 1 1 0 1 0 1 1

Llego = 0 1 0 1 1 0 1 0 0 1 1

11 10 9 8 7 6 5 4 3 2 1

agarramos las convinaciones q' hay 1 y pongo el eq. al indice en binario

1	0 0 0 1	
2	0 0 1 0	
5	0 1 0 1	Xor
7	0 1 1 1	
8	1 0 0 0	
10	1 0 1 0	
	0 0 1 1	= 3 el error esta en el bit "3"

5. Si llega igual

1	0 0 0 1	
2	0 0 1 0	
5	0 1 0 1	
7	0 1 1 1	Xor
8	1 0 0 0	
10	1 0 1 0	
3	0 0 1 1	
	0 0 0 0	= No hay error

NOTA

Unidad 4

Sistema de numeración y codigos

HOJA N° 4

FECHA

1) $K25A_{16}$ =

NOTA

2)

	XS3 A B C D	BCD D_3 D_2 D_1 D_0
1	0 0 1 1	0 0 0 0
2	0 1 0 0	0 0 0 1
	0 1 0 1	0 0 1 0
	0 1 1 0	0 0 1 1
	0 1 1 1	0 1 0 0
	1 0 0 0	0 1 0 1
	1 0 0 1	0 1 1 0
	1 0 1 0	0 1 1 1
	1 0 1 1	1 0 0 0
	1 1 0 0	1 0 0 1

$D_3 =$

AB \ CD	00	01	11	10
00				
01				
11	1			
10		1		

$D_3 = A\bar{B}\bar{C}\bar{D} + A\bar{B}CD$

$D_2 =$

AB \ CD	00	01	11	10
00				
01		(1)		
11				
10	1	1		1

$D_2 = \bar{A}BCD + A\bar{B}\bar{C} + A\bar{B}\bar{D}$

$D_1 =$

AB \ CD	00	01	11	10
00				
01		1		1
11				
10		1		1

$D_1 = \bar{A}B\bar{C}D + \bar{A}BC\bar{D} + A\bar{B}\bar{C}D + A\bar{B}C\bar{D}$

$= \bar{A}B(\bar{C}D + C\bar{D}) + A\bar{B}(\bar{C}D + C\bar{D}) = \bar{A}B(C \oplus D) + A\bar{B}(C \oplus D)$

$= (A \oplus B)(C \oplus D)$

$D_0 =$

AB \ CD	00	01	11	10
00				
01	(1)			1
11	1			
10	1			1

$D_0 = \bar{A}B\bar{D} + A\bar{B}\bar{D} + B\bar{C}\bar{D} = B\bar{C}\bar{D} + \bar{D}(\bar{A}B + A\bar{B})$

$D_0 = \bar{D}\left[B\bar{C} + (A \oplus B)\right]$

NOTA

megustas 77

♡ Martila+linda

HOJA N° 5

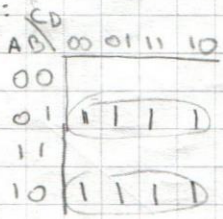
FECHA

3) Binario a gray (4 bit)

A B C D	$Z_3 Z_2 Z_1 Z_0$
0 0 0 0	0 0 0 0
0 0 0 1	0 0 0 1
0 0 1 0	0 0 1 1
0 0 1 1	0 0 1 0
0 1 0 0	0 1 1 0
0 1 0 1	0 1 1 1
0 1 1 0	0 1 0 1
0 1 1 1	0 1 0 0
1 0 0 0	1 1 0 0
1 0 0 1	1 1 0 1
1 0 1 0	1 1 1 1
1 0 1 1	1 1 1 0
1 1 0 0	1 0 1 0
1 1 0 1	1 0 1 1
1 1 1 0	1 0 0 1
1 1 1 1	1 0 0 0

$Z_3 = A$

Z_2:

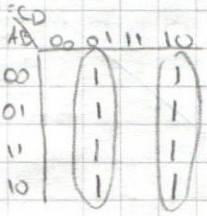


$Z_2 = \bar{A}B + A\bar{B} = A \oplus B$

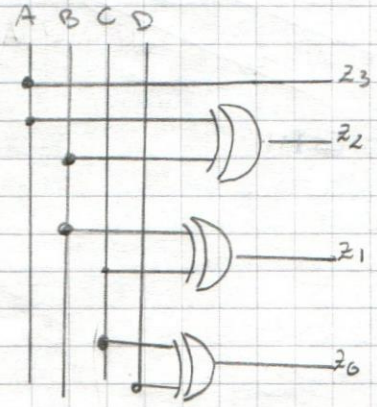
Z_1:

$Z_1 = B\bar{C} + \bar{B}C = B \oplus C$

Z_0:



$Z_0 = \bar{C}D + C\bar{D} = C \oplus D$




```
entity gray is
    port( bin: IN std_logic_vector (3 downto 0);
          gray: OUT std_logic_vector (3 downto 0);
end gray;

architecture codigo of gray is
  begin
      process (bin)
        for I in 0 To 3 loop
            gray (I) <= bin(I) xor bin (I+1);
        end loop;
        gray (4) <= bin (3);
      end process;
end codigo;
```

NOTA

4) Obtener codigo hamming

A) 1011010 $n=7$ $2^4 = 4+7+1$

$k=4$

1 0 1 1 0 1 0

B_6	B_5	B_4	k_8	B_3	B_2	B_1	k_4	B_0	k_2	k_1
1	①	1	1	0	0	0	0	0	0	0
0	0	0	⓪	1	①	1	1	0	0	0
1	①	0	0	1	①	0	0	①	1	0
1	0	1	0	1	0	1	0	①	0	1

$k_8 = 0$
$k_4 = 0$
$k_2 = 0$
$k_1 = 0$

$Tx = 1010100000$

B) 01011000 $n=8$

$k=4$

0 1 0 1 1 0 0 0

B_7	B_6	B_5	B_4	k_8	B_3	B_2	B_1	k_4	B_0	k_2	k_1
①	1	①	1	1	0	0	0	0	0	0	0
①	0	0	0	0	1	①	①	1	0	0	0
0	1	①	0	0	1	①	0	0	①	1	0
0	1	0	1	0	1	0	①	0	①	0	1

$k_8 = 0$
$k_4 = 1$
$k_2 = 0$
$k_1 = 1$

$Tx = 01010100\underline{1}001$

5) obtener y corregir con hamming (metodo matricial)

A) Dato: 1 0 1 0 1 0 1 0 0 1 1

B A 9 8 7 6 5 4 3 2 1 hexa

1	0 0 0 1
2	0 0 1 0
5	0 1 0 1
7	0 1 1 1
9	1 0 0 1
B	1 0 1 1
	0 0 1 1

Xor

error en la posicion 3 : 10101010111

B) Dato: 0 1 0 0 1 1 1 1 0 0 1

B A 9 8 7 6 5 4 3 2 1

1	0 0 0 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
A	1 0 1 0
	1 0 1 1

error en B : 11001111001

NOTA

6) Codificador y decodificador hamming de 4 bit en VHDL

$n = 4$
$2^k \geq k + n + 1$
$8 = 2^3 \geq 3 + 4 + 1$

$b_4\ b_3\ b_2\ k_4\ b_1\ k_2\ k_1$

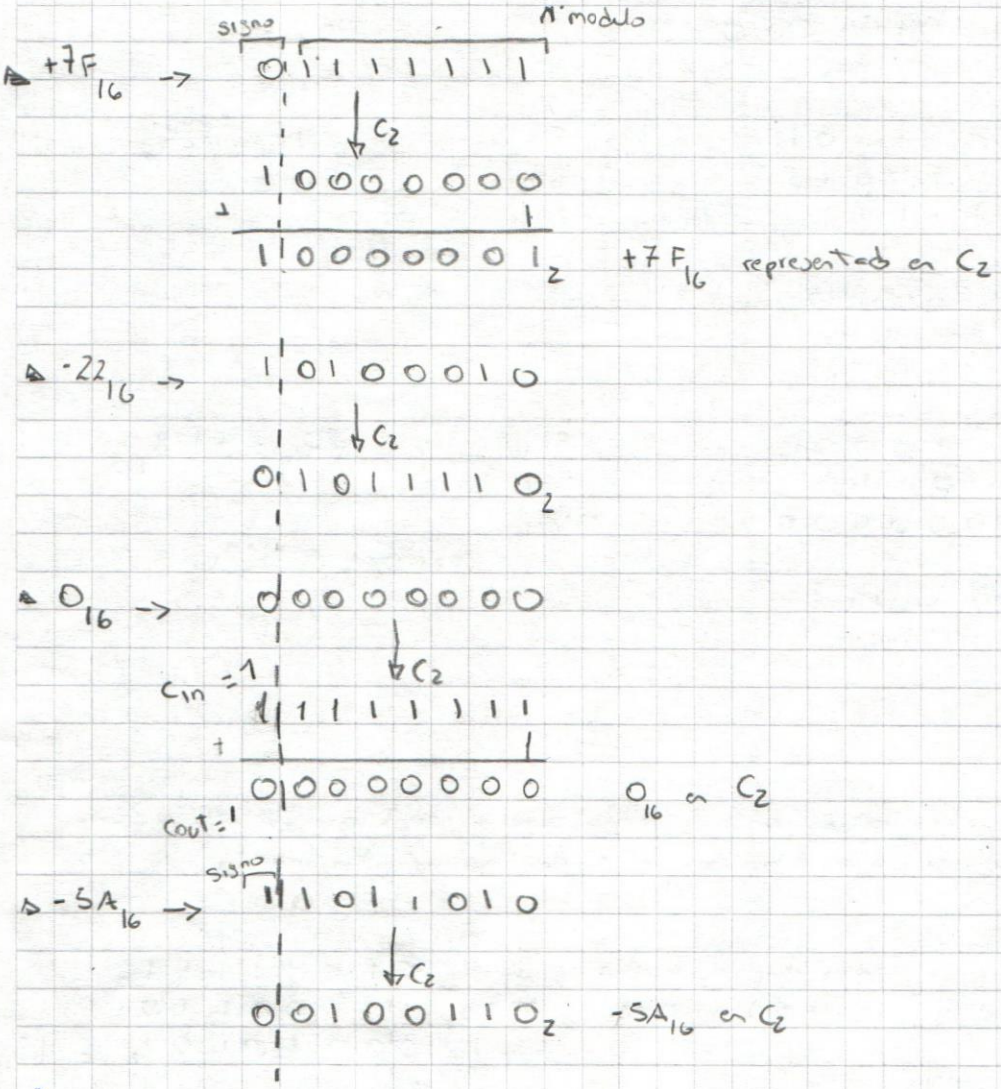
```
Entity codificador is
  port(
       b: in std_logic_vector (4 downto 1);
       k: out std_logic_vector (4 downto 1));
end codificador;

Architecture codi of codificador is
  begin(
       k(1) <= xor( b(1), b(2), b(4));
       k(2) <= xor( b(1), b(3), b(4));
       k(4) <= xor( b(2), b(3), b(4));
end codi;
```

```
Entity decodificador is
  port
       b: in std_logic_vector (4 downto 1);   k: in std_logic_vector (4 downto 1);
       sal: out std_logic_vector (4 downto 1); altseg: out std_logic_vector (7 downto 0)
end decodificador

Architecture deco of decodificador is

    Signal  G: std_logic_vector (4 downto 1);
            C: std_logic_vector (4 downto 1);
  begin(
       G(1): xor ( k(1), b(1), b(2), b(4));
       G(2): xor ( k(2), b(1), b(3), b(4));
       G(4): xor (k(4), b(2), b(3), b(4));

    with G select

       altseg <= "0110000" when "001";   -- prende el 1
                 "1101101" when "010";   -- 2
                 "1111001" when "011";
                 "0110011" when "100";
                 "1011011" when "101";
                 "1011111" when "110";
                 "1110000" when "111";
                 "0000000" when others   -- 0

-- corrector  with G select

       C <= "0001" when "001";   -- p/ corregir bit en la pos 1
            "0010" when "010";
            "0100" when "011";
            "1000" when "100";
            "0000" when others;

out(4) <= b(4) xor C(4);   out(3) <= b(3) xor C(3);
out(2) <= b(2) xor C(2);   out(1) <= b(1) xor C(1);

end deco;
```

NOTA

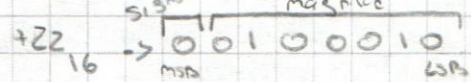
Unidad 5 | Aritmética binaria | HOJA N° 1 | FECHA

1) Los sig. numeros en hexadecimal se los expresa en C_2 en 8 bits (1 bit signo mas 7 bit p/el nº)



2) Suma con signo

Para representar un nº de 8 bit con signo ↔ 1 bit signo (msb) + 7 bits magnitud



8 bits → Sin signo: 0_{10} a 255_{10} (256 combinaciones)
→ Con signo: -128_{10} a $+127_{10}$ (")

Significa que lo max que pued representar con 8 bit con signo es +127 si n mayor produce un desborde :
overflow = $C_1 \oplus C_{out}$

NOTA

Pongo el nº de hexa en binario

▲ $30_{16} + 41_{16} = 71_{16}$

$C_1{:}0$ sign

$0\,0\,1\,1\,0\,0\,0\,0_2 + 0\,1\,0\,0\,0\,0\,0\,1_2 = 0\,1\,1\,1\,0\,0\,0\,1$; $C_0{:}0$

$+48_{10} + 65_{10} = +113_{10}$

$71_{16} = +113_{10}$

max $+7F_{16}$ ∴ no hay overflow

overflow: $C_1 \oplus C_0 = 0 \oplus 0 = 0$ ∴ no hay

max $+127_{10}$ ∴ no hay overflow

▲ $5D_{16} + 22_{16} = 7F_{16}$

$C_1{:}0$

$0\,1\,0\,1\,1\,1\,0\,1 + 0\,0\,1\,0\,0\,0\,1\,0 = 0\,1\,1\,1\,1\,1\,1\,1$; $C_0{:}0$

$93_{10} + 65_{10} = +127_{10}$

$7F_{16} = 127_{10}$

max $+7F$ ∴ no hay overflow

over $= C_1 \oplus C_0 = 0$ ∴ no hay

max $+127_{10}$ ∴ no hay

▲ $51_{16} + 2F_{16} = 80_{16}$

$C_1{:}1$

$0\,1\,0\,1\,0\,0\,0\,1 + 0\,0\,1\,0\,1\,1\,1\,1 = 1\,0\,0\,0\,0\,0\,0\,0$; $C_0{:}0$

$81_{16} + 47_{16} = +128_{10}$

$80_{16} = +128_{10}$

es mayor a $7F_{16}$ ∴ sí hay overflow

over $= C_1 \oplus C_0 = 1$ ∴ sí hay

es mayor a $+127_{16}$ ∴ sí hay

Resta con signo

- la operación C_2 permite pasar un nº negativo a positivo. de esta forma la resta se convierte en una suma de un nº positivo y otro en C_2

▲ $+4F_{16} - +3A_{16} = +15_{16}$

$+79_{10} - +58_{10} = +21_{10}$

$0\,1\,0\,0\,1\,1\,1\,1 - 0\,0\,1\,1\,1\,0\,1\,0 \xrightarrow{C_2}$ $C_1{:}1$

$0\,1\,0\,0\,1\,1\,1\,1 + 1\,1\,0\,0\,0\,1\,0\,1 = 0\,0\,0\,1\,0\,1\,0\,1_2 = 15_{16} = 21_{10}$; $C_0{:}0$ [+]

min -80_{16} ∴ no hay over

min -128_{10} ∴ no hay over

over $= C_1 \oplus C_0 = 1 \oplus 0 = 0$ ∴ no hay overflow

el rdo es (+) ∴ no hace falta aplicar de nuevo el C_2

▲ $+0_{16} - +7F_{16} = -7F_{16}$

$+0_{10} - +127_{10} = -127_{10}$

$0\,0\,0\,0\,0\,0\,0\,0_2 - 0\,1\,1\,1\,1\,1\,1\,1_2 \xrightarrow{C_2}$ $C_1{:}0$

$0\,0\,0\,0\,0\,0\,0\,0 + 1\,0\,0\,0\,0\,0\,0\,1 = 1\,0\,0\,0\,0\,0\,0\,1$; $C_0{:}0$ [-]

min $= -80_{16}$ no hay

min -128_{10} no hay

overflow $= C_1 \oplus C_0 = 0$ no hay

∴ al ser [-] aplico C_2 al rdo

$\text{rdo}|_{C_2} = 0\,1\,1\,1\,1\,1\,1\,1_2 = |7F_{16}| = |127_{10}|$

▲ $-62_{16} + -70_{16} = -D2_{10}$

$-98_{10} + -112_{10} = -210_{10}$

$-0\,1\,1\,0\,0\,0\,1\,0 + -0\,1\,1\,1\,0\,0\,0\,0 \xrightarrow{C_2}$

$+1\,0\,0\,1\,1\,1\,1\,0_2 + 1\,0\,0\,1\,0\,0\,0\,0_2$; $C_1{:}0$

$C_0{:}1$ $0\,0\,1\,0\,1\,1\,1\,0$

min -80_{16} ∴ sí hay

min -128_{10} ∴ sí hay overflow

over $= C_1 \oplus C_0 = 1$ existe overflow

NOTA

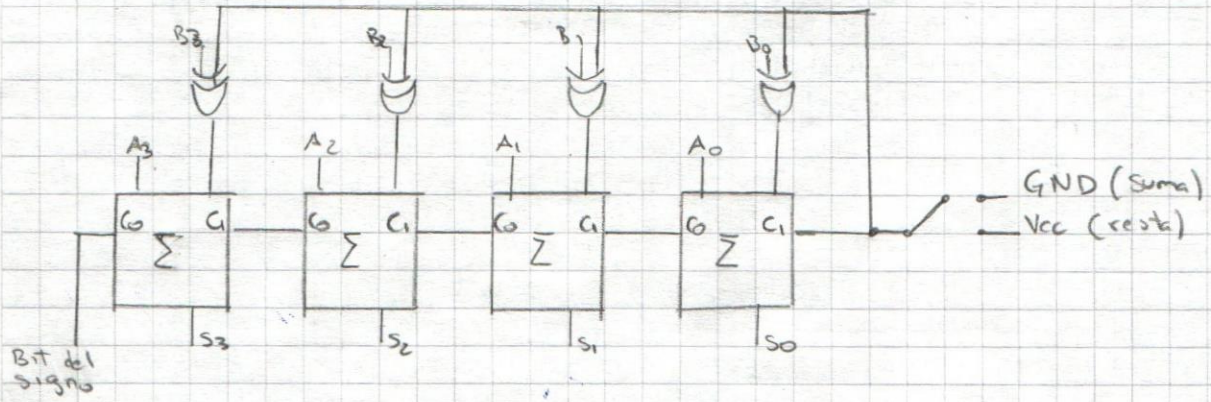
Suma { $S: C_i \oplus A \oplus B$; $C_{out}: AB \oplus CB \oplus CA$ }

Resta { $S: A \oplus B + C_{in}$; $C_{out}: A\bar{B} + \overline{C_{in}}(A \oplus B)$ }

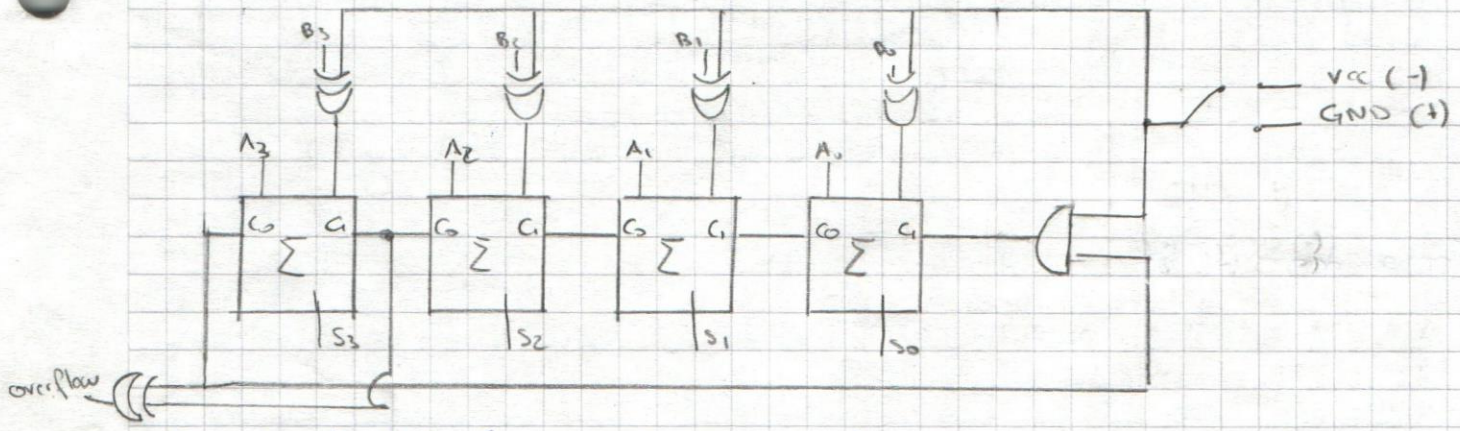
HOJA Nº 2

FECHA

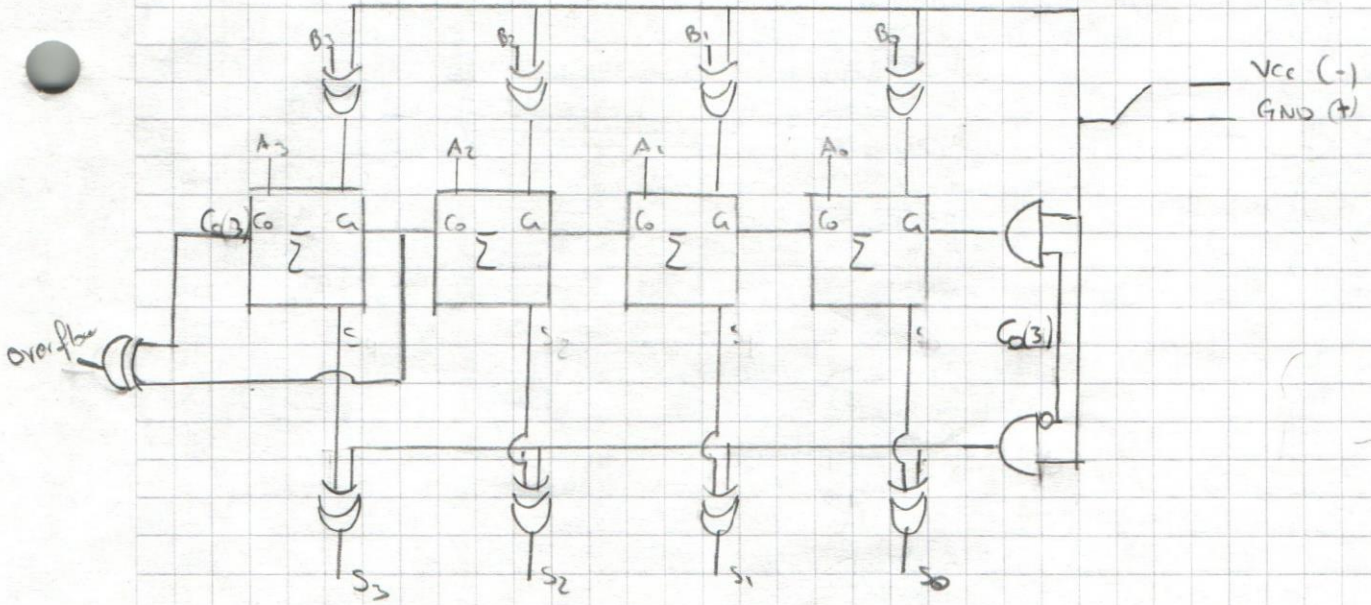
3) Sumador/restador (con signo)



Sumador/restador (sin signo y sin modo a la salida)



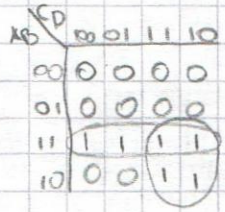
Sumador/restador (con ~~modo~~ ~~a la salida~~ Signo



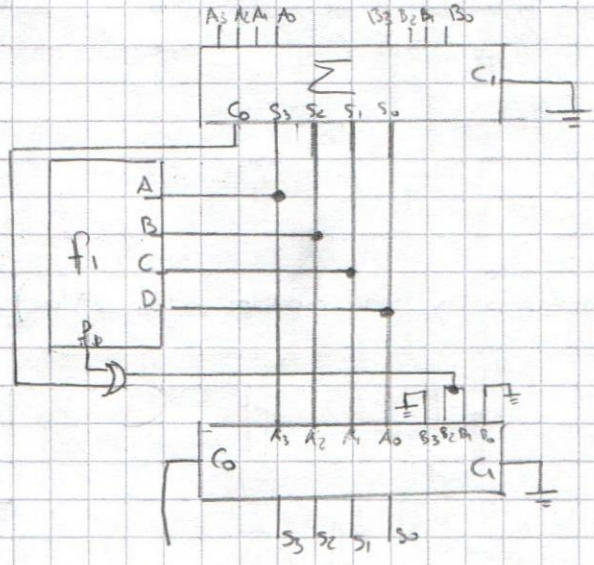
NOTA

Sumador BCD

A	B	C	D	F_1
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

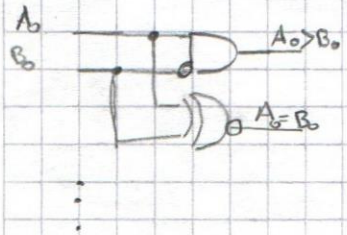


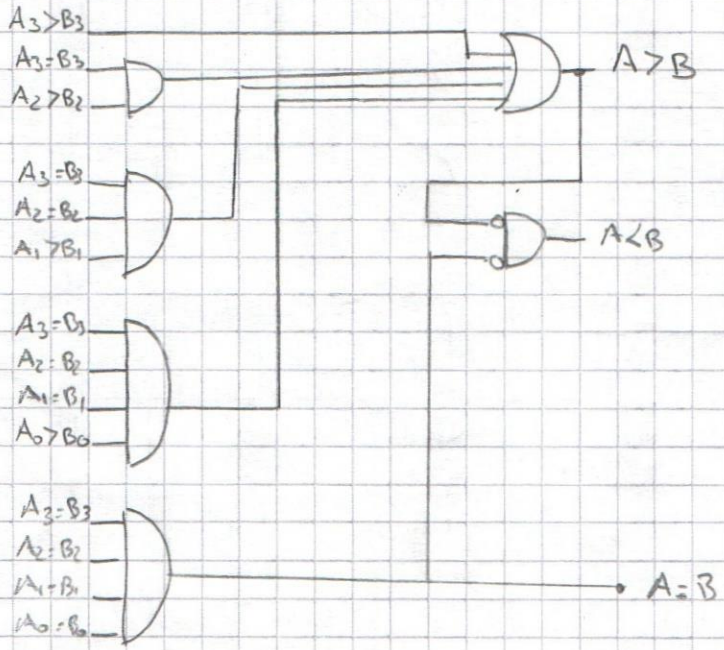
$f_1 = A.B + AC$



Comparador 4 bit

A	B	A=B	A>B	A<B
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0
		$\overline{A \oplus B}$	$A\overline{B}$	$\overline{A}B$





NOTA

4) VHDL: Sumador 16 bit

```
Use ieee.std_logic_arith.all;     -- p/ usar el (+) p/ sumar

Entity sumador is
   port(
        Cin:              : IN  std_logic;
        X, Y:             : IN  std_logic_vector (15 downto 0);
        S:                : out std_logic_vector (15 downto 0);
        Cout, overflow:   : out std_logic; );
end sumador;

Architecture sum of sumador is
   signal suma: std_logic_vector (17 downto 0);

 begin
        suma <= ('0' & x) + y + Cin;        -- Concatenación: '0' & x es p/ crear el bit 17 de overflow; si es que hay
          S <= suma(15 downto 0);
       Cout <= suma(17)
   overflow <= suma(17) xor x(15) xor y(15) xor suma(15);
end sum;
```

VHDL: Comparador (4 bit)

```
Use ieee.std_logic_arith.all;

Entity comparador is
     port(
          A, B :          : IN  std_logic_vector (3 downto 0);
          Amay, Igual, Amen:  out  std_logic_ );
end comparador

Architecture comp of comparador is
 begin
      Amay <= '1' when A>B else '0';
     Igual <= '1' when A=B else '0';
     Amenor <= '1' when A<B else '0';
end comp;
```

NOTA

NOTA

RTL hardware desing using VHDL (chu)

Bombillo → ayuda VHDL
Síntesis – condicionales

RTL schematic (p/ ver q' interpreta)

HOJA N° 1

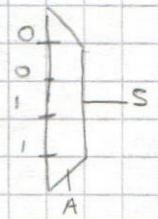
FECHA

Circuito Combinacional (VHDL)

Sentencia de selección de señal (with select)

Trabaja como un mux

```
with A select
   S <= '0' when "00",
        '0' when "01",
        '1' when "10",
        '1' when others;
```



#. Sentencia de asignación condicional (when-else)

Se puede hacer lo mismo q' with select pero la implementación física es distinta

```
S <= Valor1 when cond1 else
     Valor2 when cond2 else
     Valor3 ;
```

Si la cond1 es verdadera, le asigna a S cond1; sino lo es, evalua la cond2; si es verdadera se la asigna a 'S' y así...

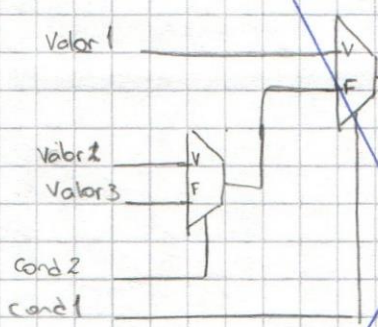
" Evalua con prioridad "

con with select

```
sel <= m&n;
with sel select
   S <= a+b when "00",
        a+b when "11",
        a or b when "10",
        a and b when others;
```

con when else

```
S <= a+b when m=n else
     a or b when m>n else
     a and b ;
```



sel <= m&n

sig q' "00"
m <= n

Aritmética

medio { Suma = A xor B
Cout = A and B

Completo { Suma = (A xor B) xor Cin
Cout = (A and B) or [(A xor B) and Cin]

medio { resta = A xor B
cout = $\bar{A}$ and B

Completo { resta = $(A\bar{B}\bar{C_{in}}) + (\bar{A}B\bar{C_{in}}) + (\bar{A}\bar{B}C_{in}) + (ABC)$ = $A \oplus B \oplus C_{in}$
cout = $(\bar{A}C_{in}) + (\bar{A}B) + (B \cdot C_{in})$

NOTA

Componente (instanciación) (Sumador de 4 bits)

- pasar un archivo a "alto nivel" o hacerlo "top" → set as top module (xq' eso no lo podemos sintetizar)

- Se crean en VHDL module los componentes (mira video 4 carlos fajardo)

- Instanciar: es agregar el comp. al archivo de "alto nivel"

- Component sirve p/ agregar el "sumador medio" al programa principal

```
Entity sumador_4bit_alto_nivel is
  port ( Ent1: in std_logic_vector (3 downto 0)
         Ent2: in   "    "      "
    resultado: out  "    "      4
        carry: out std_logic ); end sumador_4bit_alto_nivel;

Architecture Bena of sumador_4bit_alto_nivel is

component sumador_medio                    // los componentes los declaro antes
  port (                                      del begin (q' previamente en Xilinx
        A : in  std_logic;                    los cree con VHDL module) y
        B : in   "
     Cout: out   "
     suma: out   "    ); end component;

component sumador_completo
  port (
        A : in  std_logic;
        B : in   "
       Cin: in   "
      Cout: out  "
     suma : out  "    ); end component;

Signal C1, C2, C3 : std_logic                // necesito señales en el sumador de 4bit
                                               q' son el Cout q' entra a otro sumador
       (Cn)

begin
   nombre                  componente q' esta instanciado
inst_sumador_medio : sumador_medio port map(          // instanciacion

    A => Ent1(0),
    B => Ent2(0),
 Cout => C1,
 suma => resultado(0));
```

NOTA

HOJA N° 2

FECHA

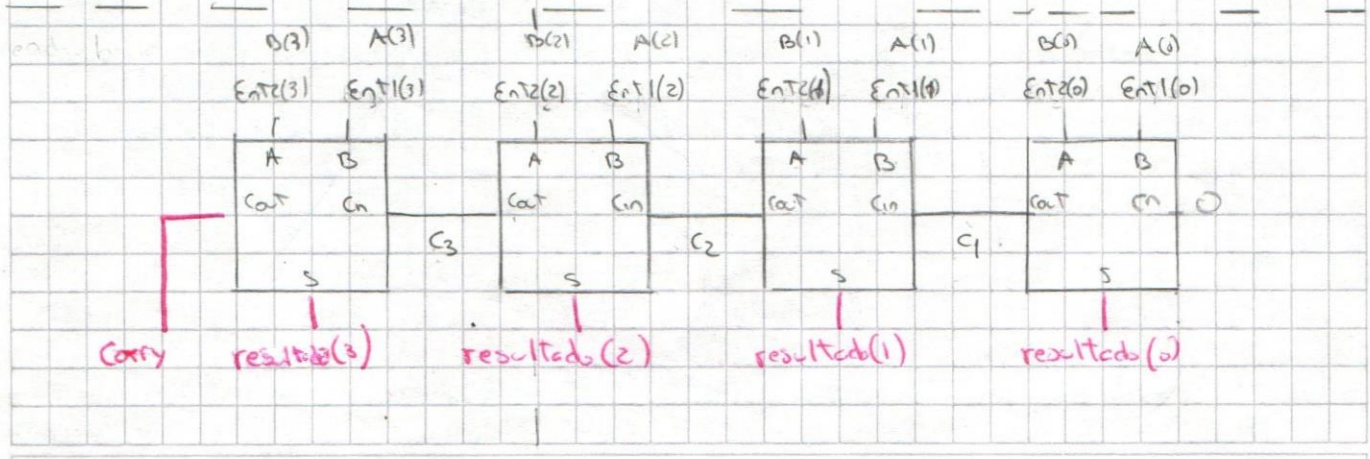
```
inst_sumador_completo1: sumador_completo port map(
        Cn => C1,
        A => Ent1(1),
        B => Ent2(1),
        Cn => C1,
        Cout => C2,
        suma => resultado(1) );

inst_sumador_completo2 : sumador_completo port map(
        A => Ent1(2),
        B => Ent2(2)
        Cn => C2,
        Cout => C3,
        suma => resultado(2) );

inst_sumador_completo3 : sumador_completo port map(
        A => Ent1(3),
        B => Ent2(3),
        Cn => C3
        Cout => Carry
        suma => resultado(3);

end beha;
```

// como en un sumador de 4 bit uso 3 Σ completos; lo instancio 3 veces con dist nombre

// Ahora desarrollo el comportamiento de los componentes

```
Entity sumador_medio is
  port(
      A, B: in std_logic;
   suma, cout : out  " );
end sumador_medio;

Architecture beha of sumador_medio is
begin
    cout <= A and B;
    suma <= A xor B;  end beha;
```

```
Entity sumador_completo is
port(
        A, B, Cn : in std_logic;
    suma, cout : out  " ); end sumador_completo;

Architecture beha of sumador_completo is
begin
    cout <= (A and B) or ((A xor B) and Cin);
    suma <= (A xor B) xor Cin;
    end beha;
```



NOTA

Circuito Secuencial en cuanto uso process

Process con lista sensible

// después del begin de architecture

// Cuando una señal o entrada que se encuentre en la lista sensible, cambia el process se activa

// Las instrucciones se ejecutan secuencialmente

```
process (a,b,c)
begin
    y <= a and b
    y <= a or c
    y <= a and c
end process
```

=>

```
process(c,b,c)
begin
    y <= a and c;
end process
```

// Xq' al ser secuencial significa q' y = a & última asignación

lista sensible

A)
```
Process ( A, B, C, D , seleccion) begin

  if seleccion = "00" then
      salida <= A;
  elsif seleccion = "01" then
      salida <= B;
  elsif seleccion = "10" then
      salida <= C;
  else
      salida <= 'D' ;
  end if;

end process
```

If/else

B)
```
Process (A, B, C, D, seleccion) begin

  case seleccion is
      when "00" => salida <= A;
      when "01" => salida <= B;
      when "10" => salida <= C;
      when "11" => salida <= D;
  // when others => salida = 0; // puede servir pero en max no va

  end case

end process
```

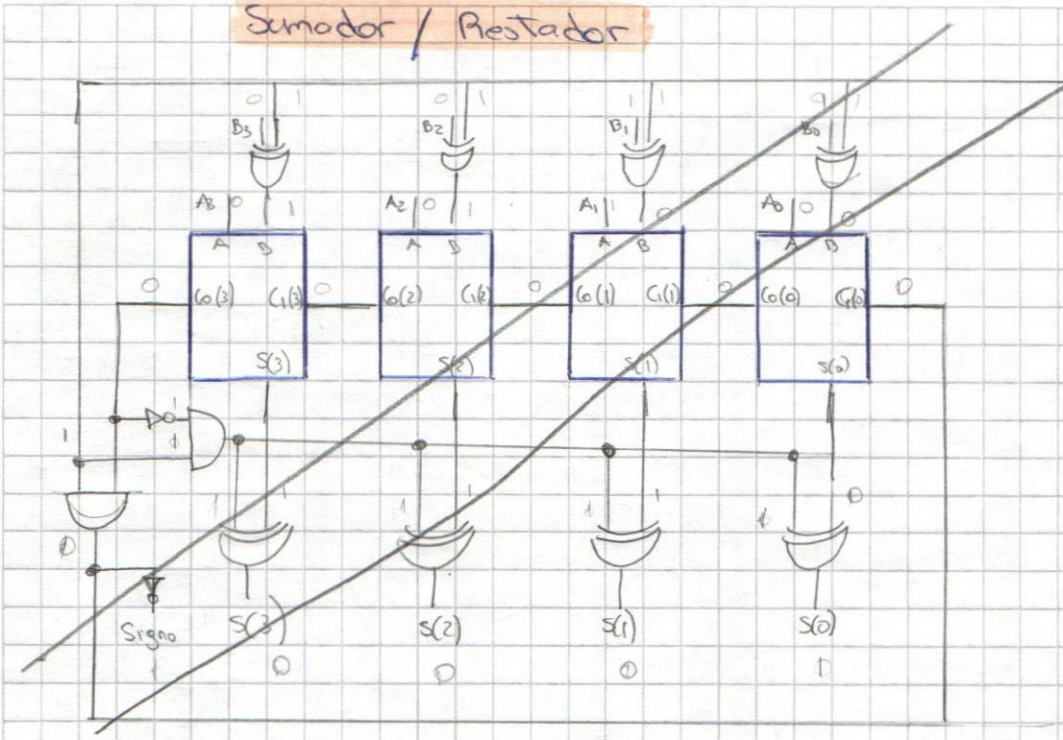
Case

NOTA

HOJA N°

FECHA

Sumador / Restador



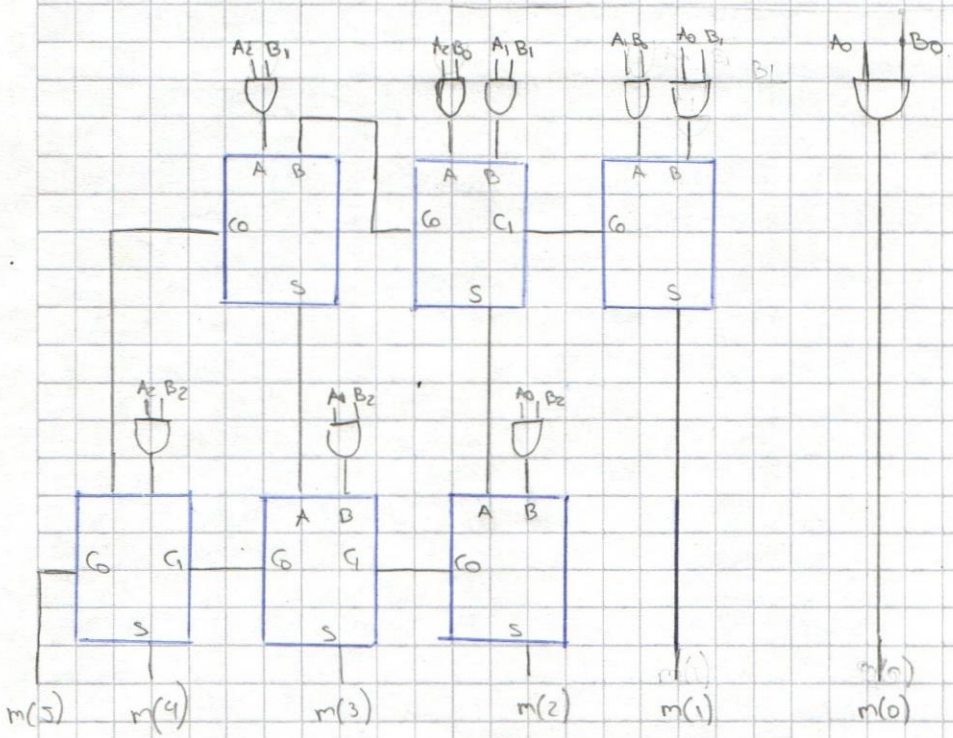
2 A 0010

-3 B 0011

Signo = 0 (+)

1 (-)

Multiplicador 3 bit con 3 bit



	X	A_2	A_1	A_0	
		B_2	B_1	B_0	
		A_2B_0	A_1B_0	A_0B_0	
	A_2B_1	A_1B_1	A_0B_1		
Cat	A_2B_2	A_1B_2	A_0B_2		
m(5)	m(4)	m(3)	m(2)	m(1)	m(0)

NOTA

NOTA

1)

A)

A	B	C	f_1
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

f_1

A \ BC	00	01	11	10
0	1	1	1	0
1			1	

$f_1 = \bar{A}\bar{B} + BC$

E	D	f_A
0	0	0
0	1	0
1	0	f_1
1	1	1

f_A

E \ D	0	1
0		
1	f_1	1

$f_A = E D + E \bar{D} f_1 =$

$f_A = E D + E \bar{D} (\bar{A}\bar{B} + BC)$

B)

A	B	f_1	f_2
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	1

f_1

A \ B	0	1
0	1	1
1		

$f_1 = B$

f_2

A \ B	0	1
0		
1	1	1

$f_2 = A$

D	C	f_3
0	0	0
0	1	D
1	0	f_1
1	1	f_2

$f_3 =$

D \ C	0	1
0	0	0
1	f_1	f_2

$f_3 = D\bar{C} f_1 + DC f_2 = \overline{(\bar{C}+C)}(D f_1 + D f_2)$ $\quad \bar{C}+C = 1$

C)

A	B	f_1
0	0	0
0	1	1
1	0	A
1	1	0

f_1

A \ B	0	1
0	0	1
1	A	0

$f_1 = A\bar{B}A + \bar{A}B =$
$A\bar{B} + \bar{A}B$
$= A \oplus B$

E	f
0	f_2
1	f_1

F =

$F_C\big|_{E=0} = B + \bar{D} + \bar{B}AD$

$F_C\big|_{E=1} = A \oplus B + \bar{D} + \bar{B}AD$

B	C	f_2
0	0	0
0	1	0
1	0	1
1	1	1

f_2

B \ C	0	1
0	0	0
1	1	1

$f_2 = B$

B	D	f_3
0	0	1
0	1	A
1	0	1
1	1	0

f_3

B \ D	0	1
0	1	A
1	1	0

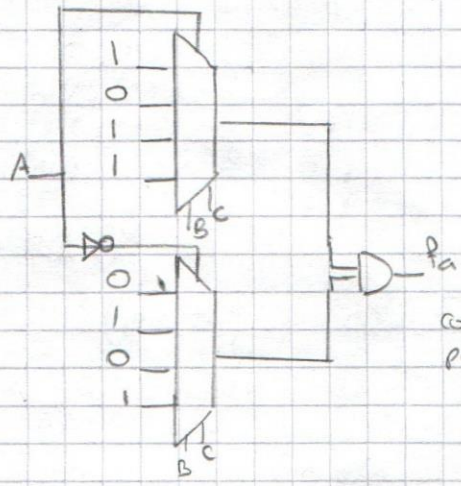
$f_3 = \bar{D} + \bar{B}AD$

NOTA

2) Resolver las func con 2 mux

$f_a = \Sigma(0,2,3,5,7)$ mux1 - mux1 $\varepsilon = 1$ $out = 1$

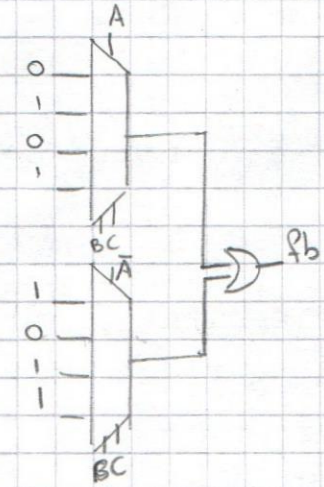
A	B	C	f_a
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



como la salida del [mux 1] es 1 cuando $\varepsilon = 1$ por eso pongo and

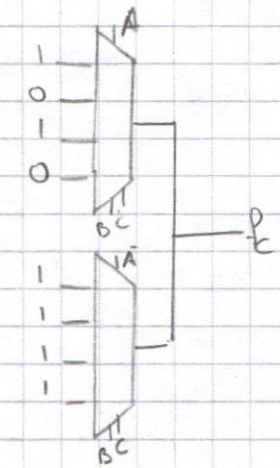
$f_b = \Sigma(1,3,4,6,7)$ [mux2 - mux2] $\varepsilon = 0$ $out = 0$

A	B	C	f_b
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



$f_c = \bar{\Sigma}\ 0,2,4,5,6,7$ $\varepsilon = 1$ $out = \bar{z}$

A	B	C	f_c
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

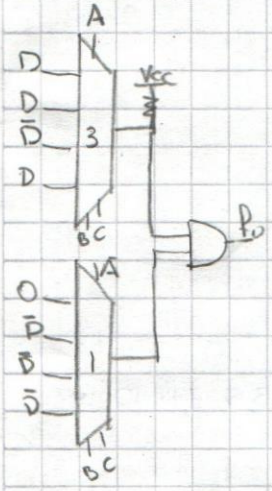


NOTA

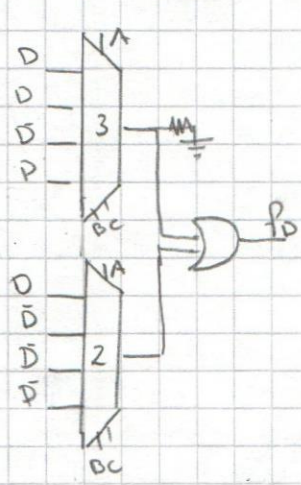
$P_D = \Sigma 1,3,4,7,10,12,14$ f_1 (mux3 - mux1)

A	B	C	D	f	f_1
0	0	0	0	0	
0	0	0	1	1	
0	0	1	0	0	Mux
0	0	1	1	1	3
0	1	0	0	1	
0	1	0	1	0	
0	1	1	0	0	
0	1	1	1	1	—
1	0	0	0	0	
1	0	0	1	0	
1	0	1	0	1	
1	0	1	1	0	Mux
1	1	0	0	1	1
1	1	0	1	0	
1	1	1	0	1	
1	1	1	1	0	—

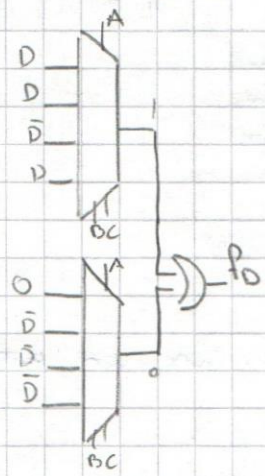
mux3 - mux1



mux3 - mux2



mux1 - mux2



NOTA

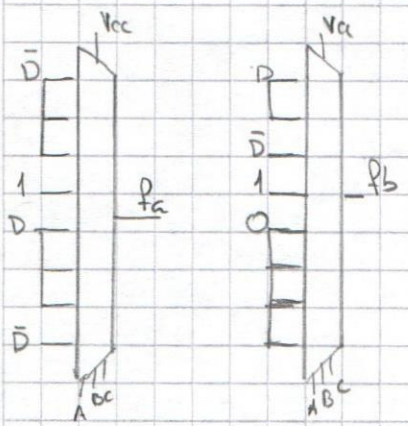
P/ mux de 8 entradas sólo importa D, p/ uno de 4 importa CD, p/ uno de 2 importa BCD

3) Implementar las func usando mux 8a1

$f_a = \Pi 0, 2, 3, 5, 7, 12, 10, 14$

$f_b = \Sigma 1, 3, 4, 6, 7$

A	B	C	D	f_a	f_b
0	0	0	0	1	0
0	0	0	1	0	1
0	0	1	0	1	0
0	0	1	1	0	1
0	1	0	0	1	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	1	0
1	1	1	1	0	0



4) Implementar las func. con mux 4a1

$f_a = \Sigma 0, 2, 3, 5, 7, 12, 10, 14$

A	B	C	D	f_a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

D_0 D_1 D_2 D_3

f_a

CD \ AB	00	01	11	10
00	1	0	1	1
01	0	1	1	0
11	1	1	1	1
10	0	0	1	0

$D_0 = C + \bar{D}$

CD \ AB	00	01	11	10
00	0	1	0	0
01	1	0	1	1
11	0	1	0	0
10	1	1	0	1

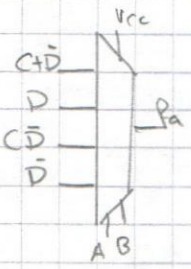
$D_1 = D$

CD \ AB	00	01	11	10
00	0	0	0	0
01	1	1	0	1
11	1	1	0	1
10	0	0	0	0

$D_2 = C\bar{D}$

CD \ AB	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	1	1	1	1
10	0	0	0	0

$D_3 = \bar{D}$



NOTA

E=0 out = ABCD
E=1 out = "2"

a
f | g | b
e | d | c

5) Decodificador BCD a 7 seg. con mux 4a1

enable		A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	0	1	1	1	1	1	1	0
0	1	0	0	0	1	0	1	1	0	0	0	0
0	2	0	0	1	0	1	1	0	1	1	0	1
0	3	0	0	1	1	1	1	1	1	0	0	1
0	4	0	1	0	0	0	1	1	0	0	1	1
0	5	0	1	0	1	1	0	1	1	0	1	1
0	6	0	1	1	0	1	0	1	1	1	1	1
0	7	0	1	1	1	1	1	1	0	0	0	0
0	8	1	0	0	0	1	1	1	1	1	1	1
0	9	1	0	0	1	1	1	1	0	0	1	1
1		1	0	1	0	X	X	X	X	X	X	X
1		1	0	1	1	X	X					
1		1	1	0	0	X	X					
1		1	1	0	1	X	X					
1		1	1	1	0	X	X					
1		1	1	1	1	X	X					

a	b	c	d	e	f	g
C+D̄	1	C̄+D	C+D̄	D̄	C̄D̄	C
C+D	C⊕D	1	CD̄	CD̄	C̄+D̄	C̄+D̄
1	1	1	D̄	D̄+C	1	1
				Red q- este rel		
X	X	X	X	X	X	X
pongo a cero Todo						

▲ P/ segmento (a)

	D=0	D=1
C=0	1	0
C=1	1	1

$D_0 = \bar{D} + C$

	D=0	D=1
C=0	0	1
C=1	1	1

$D_1 = C + D$

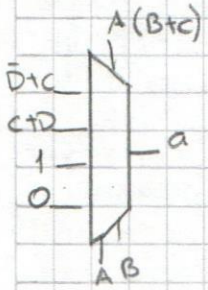
	D=0	D=1
C=0	1	1
C=1	X	X

$D_3 = 1$

▲ P/ enable

AB \ CD	00	01	11	10
00				
01				
11	1	1	1	1
10	0	0	1	1

enable = AB + AC = A(B+C)



NOTA

6) VHDL: mux 4:1 usando when-else con control de salida p/ tercer estado (es decir con enable)

```
Entity mux is
  port ( dato : in std_logic_vector(4 downto 1);
         enable : in std_logic;
         salida : out std_logic;
         sel : in std_logic_vector (2 downto 1); );
end mux;

Architecture mux4a1 of mux is
  begin

    salida <= 'z' when enable = '0' else
              dato(1) when ( sel = "00" and enable = '1' ) else
              dato(2) when ( sel = "01" " " "
```

VHDL : mux 8a1 de dos 8bits usando 2 mux 4a1 con instanciación

```
Entity mux8a1 is
  port ( Enable : in std_logic
         Ent : in std_logic_vector (8 downto 1);
         select : in std_logic_vector (2 downto 1);
         sal : out std_logic );
end mux8a1

Architecture M of mux8a1 is
  Signal S1, S2 : std_logic ;          -> salidas de c/ mux, por eso son señales

-- me refiero al primero del punto 6 -> Component mux
                port( dato : in std_logic_vector (4 downto 1)
                      enable : in std_logic
                      set : in std_logic_vector (2 downto 1) );
  end component       Salida : out std_logic

begin
    Inst_mux4a1_A : mux
      port map (
                dato(4) => Ent(8)      enable => Enable
                dato(3) => Ent(7)      sel(1) => select(1)
                dato(2) => Ent(6)      sel(2) => select(2)
                dato(1) => Ent(5)      S1 <= S );     -> no se si esto ba

    Inst_mux4a1_B : mux
      port map (
                dato(4) => Ent(4)      enable => Enable
                dato(3) => Ent(3)      sel(1) => select(1)
                dato(2) => Ent(2)      sel(1) => select(2)
                dato(1) => Ent(1)      S2 <= S );     -> Tampoco se

  sale S1 or S2 ;
```

NOTA

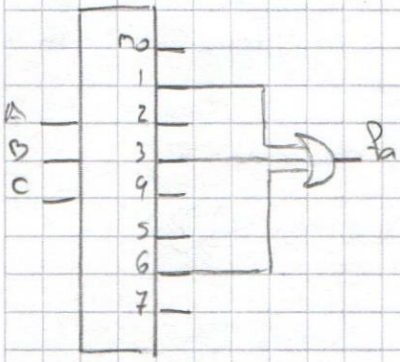
8) Resolver las funciones usando un codificador 3 a 8

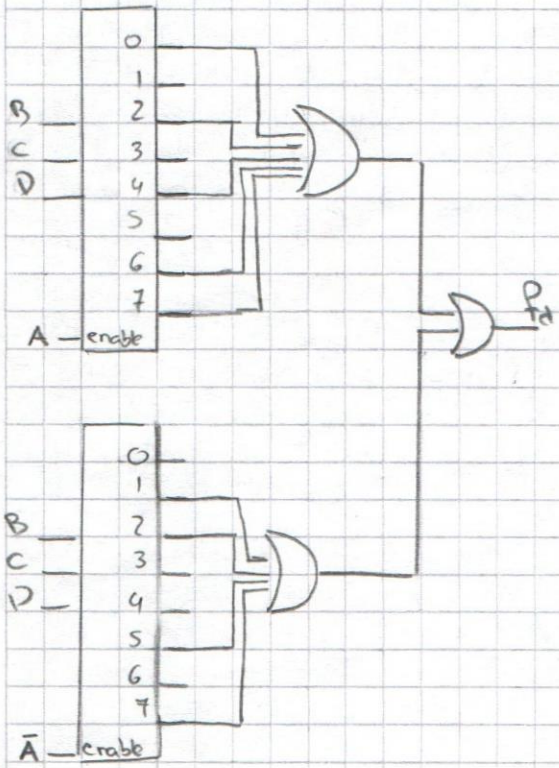
A	B	C	f_a
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

A	B	C	D	f_d
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

$f_a = \Pi 0,2,3,5,7$

$f_d = \Pi 1,3,4,7,10,12,14$





NOTA

YHDL codificador 4a2 con prioridad usando when-else (sin proceso)

```
Entity codificador is
  port (
        ent : in  std_logic_vector (3 downto 0)
        sal : out  "  (1 downto 0)
        z : out std_logic );
end codificador

Architecture codi of codificador is
  begin

      sal <= "11" when (ent(3) = '1' and z = '1') else
             "10"  "   (ent(2) = '1'  "    "    "
             "01"  "   (ent(1) = '1'  "    "    "
             "00"  "   (ent(0) = '1'  "    "    "
             "xx"  others

end codi
```

E_3	E_2	E_1	E_0	S	s	z
0	0	0	0	x	x	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

9) decodificador de errores p/ un hamming de 9 bits utilizando codificadores 3:8

no se como se hace

$b = 9 \quad 2^4 = 4+1+9$

$\therefore h = 4 \quad 16 = 14$

NOTA

VHDL decodificador 2:4 con when-else (sin process)

```
Entity decodificador is
    Port ( ent    : in  std_logic_vector (1 downto 0);
           enable : in  std_logic
           salida : out std_logic_vector (3 downto 0));
end decodificador

Architecture of decodificador is deco
 begin
      salida <= "0001" when (ent="00" and enable='1') else
                "0010"  "    "  = "01"  "     "    ) "
                "0100"  "    "  = "10"  "     "    )
                "1000"  "    "  = "11"  "     "    ) "
                "0000" others;

end deco
```

VHDL decodificador 2:4 con if-else y process

```
Architecture of decodificador is deco
 begin
     Process ( ent, enable)
      begin
          If (enable = '1') then
                    case ent is
                          when "00" => salida <= "0001".
                          when "01" => "  <= "0010"
                            "10" => "  <= "0100",
                            "11" => "  <= "1000".
                    end case
          elsif ( enable = '0') then
              salida <= "0000";

          end if
      end process
end deco.
```

VHDL decodificador BCD a 7segmentos con case-when (con process)

```
Entity deco is
    Port ( ent : in std_logic_vector (4 downto 0);
           salida out std_logic_vector (7 downto 0));
end deco
Architecture of deco is deco_A
 begin
     Process (ent)
      begin
          case ent is
                when "0000" => salida <= "1111110"     --n: 0
                when "0001" => salida <= "0110000"     --n: 1
                      :
                when "1001" => salida <= "1111011"     --n=8
          end case
      end process
end deco_A
```

NOTA

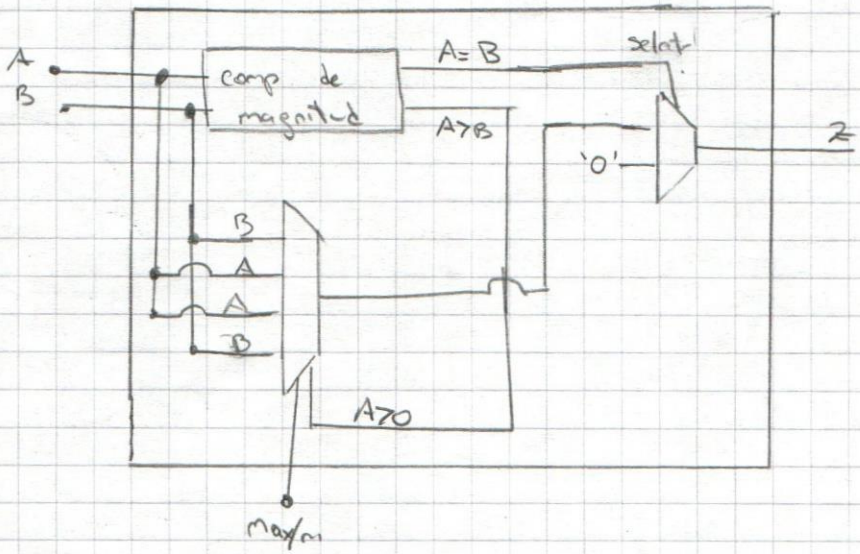
VHDL decodificador DCD a 7 seg con when-select (con process)

```
Architecture of deco is deco_b
begin
        whit (ent) selec
                salida <= "11111110" when "0000"      -- n=0
                .
                .
                .
                salida <= "111000"" when "1001"       -- n=9
end deco_b
```

b)

A=B	Max/m	A>B	Z
0	0	0	B
0	0	1	A
0	1	0	A
0	1	1	B
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Max/m : 1 Z=mm



NOTA

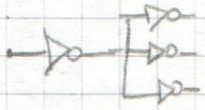
NOTA

Tecnología

HOJA Nº 1

FECHA

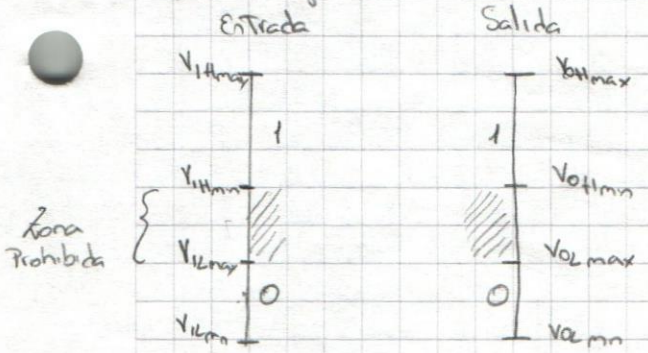
Fan out: cuantas compuertas puedo conectar sin alterar la salida



Ruido: se lo considera a cualquier Tensión que sume desfavorablemente a la Tensión de salida de un circuito lógico (sin importar su origen) y puede causar errores en el func lógico

Margen de ruido (se da a max carga)
Es la tolerancia en la tensión de entrada, que al superarse el fabricante no garantiza los niveles lógicos

Niveles lógicos

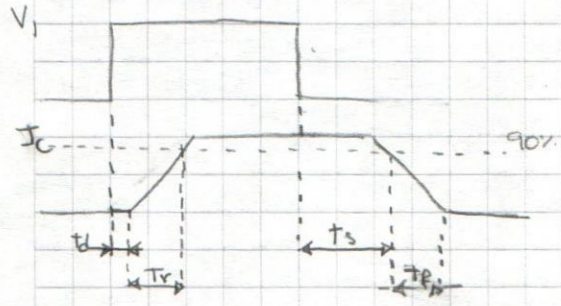


	TTL Entrada	TTL Salida	Cmos Entrada	Cmos Salida
V_{IHmax} :	5	5	5	5
V_{IHmin} :	2	2,4	3,5	4,9
$V_{OL\,max}$:	0,8	0,4	1,5	0,1
$V_{OL\,min}$:	0	0	0	0

$V_{mL}: V_{IL(max)} - V_{OL(max)}$

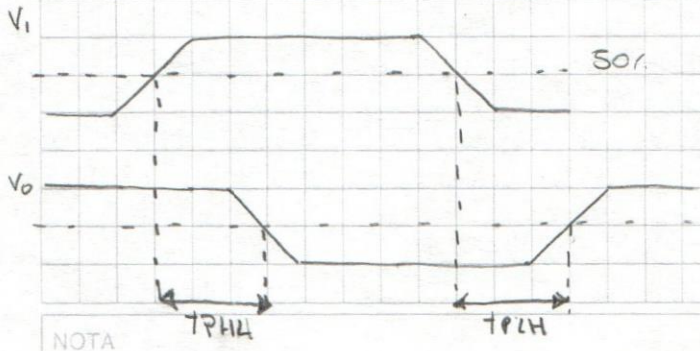
$V_{mH}: V_{OH(min)} - V_{IH(max)}$

Tiempo de Transición



- t_d (Tiempo de retardo): Tiempo requerido p/ cargar las capacidades de la unión p/ que el dispositivo pase de la zona de corte a activación
- t_r (Tiempo de subida): Tiempo en el que la I_C se eleva del 10% al 90%
- t_s (Tiempo de almacenamiento): depende de la carga de exceso de la base, mientras mas saturado esté, mas carga en exceso tendrá
- t_f (Tiempo de caída): Tiempo en el que I_C cae del 90% al 10%

Tiempo de propagación



$$T_{pd} = \frac{T_{pHL} + T_{pLH}}{2}$$

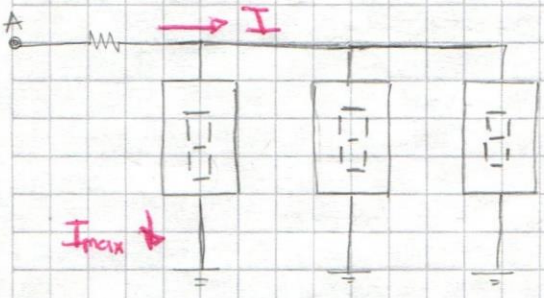
NOTA

$f_{FO\,max} = \frac{1}{T_2}$

$P_D = I_{CC} V_{CC} = \left(\frac{I_{CCH} + I_{CCL}}{2}\right) V_{CC}$

NOTA

Mux de display 7 segs.

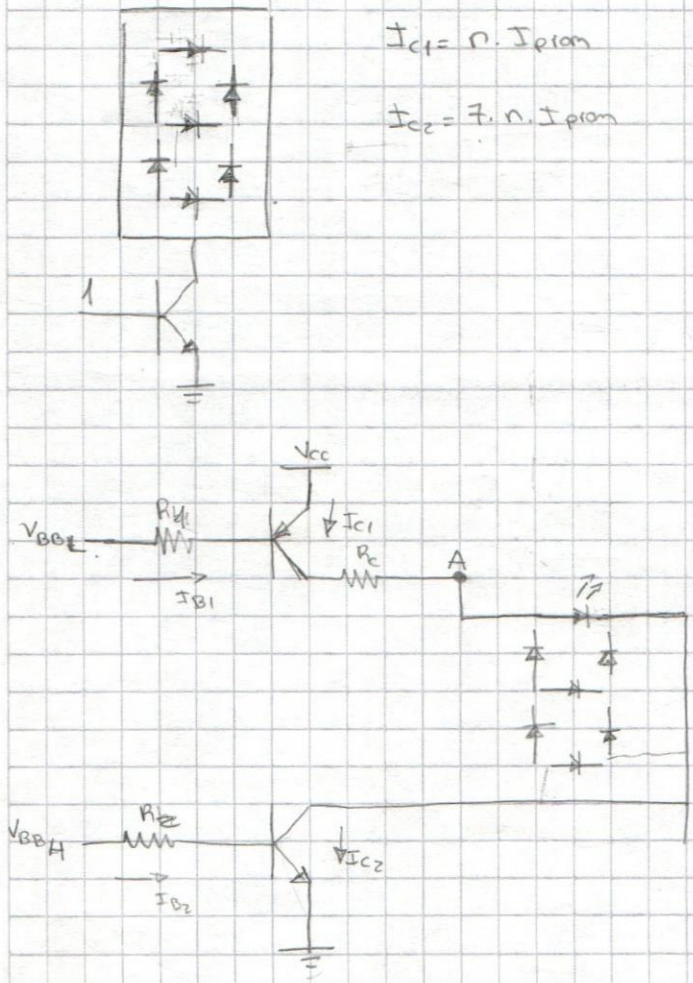


$I = n \cdot I_{prom}$

$I_{max} = 7 \cdot n \cdot I_{prom}$

n = n° de display
I_{prom} = I prom de segmento

Catodo Comun



$I_{C1} = n \cdot I_{prom}$

$I_{C2} = 7 \cdot n \cdot I_{prom}$

$V_{cc} - V_{CE} - V_{Rc} - V_{led} - V_{CE} = 0$

$$R_c = \frac{V_{cc} - 2V_{CE} - V_{led}}{I_{C1}}$$

$V_{cc} - V_{BE} - V_{R1} - V_{BEL} = 0$

$$R_1 = \frac{V_{cc} - V_{BE} - V_{BEL}}{I_{B1}}$$

$V_{BB_H} - V_{R2} - V_{BE} = 0$

$$R_2 = \frac{V_{BB_H} - V_{BE}}{I_{B2}}$$

$$I_{B1} = \frac{I_{C1}}{\beta} = \frac{n \cdot I_{prom}}{\beta}$$

$$I_{B2} = \frac{I_{C2}}{\beta} = \frac{n \cdot 7 \cdot I_{prom}}{\beta}$$

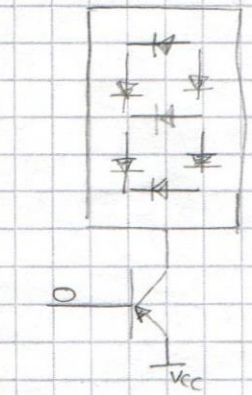
$P_{R1} = R_1 I_{B1}^2$

$P_{R2} = R_2 I_B^2$

$P_{Rc} = R_c I_{C1}^2$

NOTA

Anodo Comun



$I_{C1} = I_{pton} \cdot n$

$I_{C2} = I_{pton} \cdot n \cdot 7$

$I_{B1} = \frac{I_{C1}}{\beta}$

$I_{B2} = \frac{I_{C2}}{\beta}$

$V_{CC} - V_{CE} - V_{led} - V_{RC} - V_{CE} = 0$

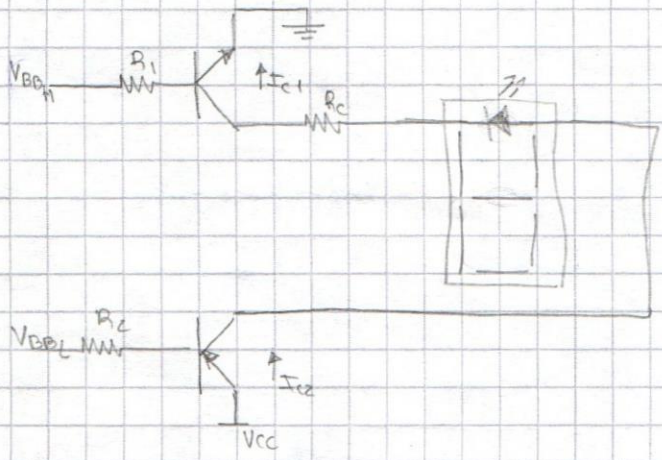
$$R_C = \frac{V_{CC} - 2V_{CE} - V_{led}}{I_{C1}}$$

$V_{CC} - V_{BE} - V_{R_2} - V_{BEOL} = 0$

$$R_2 = \frac{V_{CC} - V_{BE} - V_{BEOL}}{I_{B2}}$$

$V_{BB_H} - V_{R_1} - V_{BE} = 0$

$$R_1 = \frac{V_{BB_H} - V_{BE} -}{I_{B1}}$$



$P_{B1} = R_1 I_{B1}^2$

$P_{R2} = R_2 I_{B2}^2$

$P_{RC} = R_C I_{C1}^2$

I_{C1} y I_{C2} seria distinto

NOTA

Unidad 7

Circuitos Secuenciales

HOJA N° 1

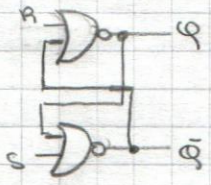
FECHA

Son circuitos lógicos realimentados.

- Latches (Biestables)

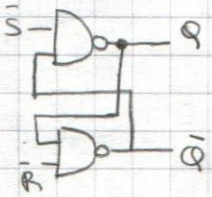
SR Asíncrono

NOR



t		t+1
R	S	Q_{t+1}
0	0	Q_t
0	1	1
1	0	0
1	1	⊗

$Q(t+1) = [\overline{(Q+S)} + R]_t$ — no permitido



NAND

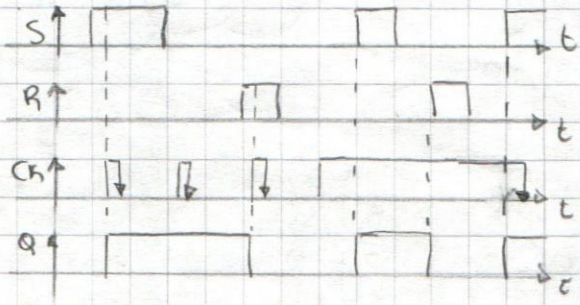
t		t+1
R	S	Q_{t+1}
0	0	⊗
0	1	1
1	0	0
1	1	Q

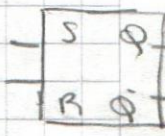
$Q(t+1) = (Q \cdot \bar{R} + S)_t$

SR Síncrono (Xq sigue una señal de reloj)

Q_t	Q_{t+1}	S	R
0	0	0	x
0	1	1	x
1	0	0	1
1	1	x	0

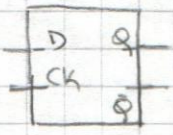
t		t+1
R	S	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	⊗



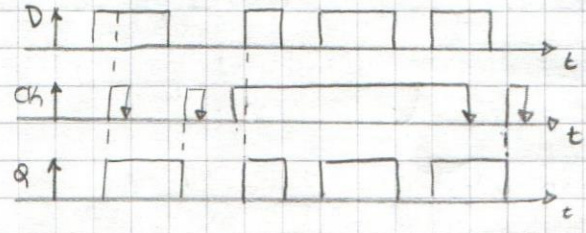


- Mientras Ck=1, Q toma los cambios en S o R
- Cuando hay un flanco de bajada en Ck, Q retiene su estado

- D síncrono (copiador; mientras el Ck=1 la salida tendremos lo mismo que la entrada el cual se memoriza cuando el Ck=0)

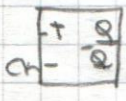


t		t+1
D		Q_{t+1}
0		0
1		1



- T (Visualizar)

t	Q_{t+1}
0	Q
1	Q̄



NOTA

-Flip-flop (permite la conexión en cascada sin tener en cuenta las restricciones impuestas por la duración del pulso de reloj)

JK con entradas asincronas

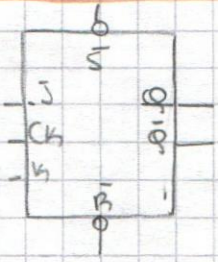


Tabla de verdad

$\bar{S}$	$\bar{R}$	J	K	Q_{t+1}	
1	1	0	0	Q_t	Sincrono
1	1	0	1	0	
1	1	1	0	1	
1	1	1	1	$\bar{Q}_t$	
0	1	x	x	1	Asincrono
1	0	x	x	0	

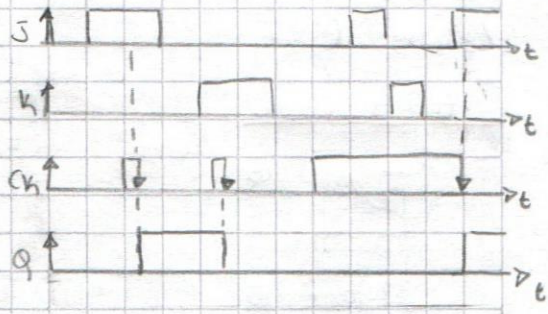
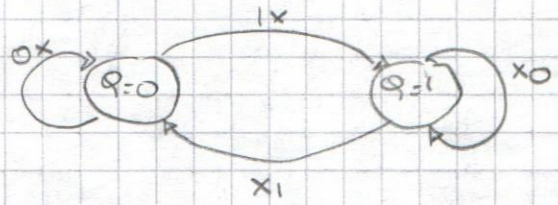


Tabla de estados

Q_t	Q_{t+1}	J K
0	0	0 x
0	1	1 x
1	0	x 1
1	1	x 0

Diagrama de los estados



D

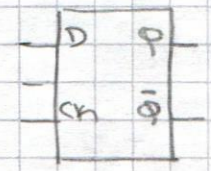


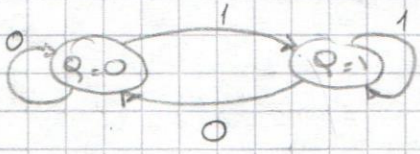
Tabla de verdad

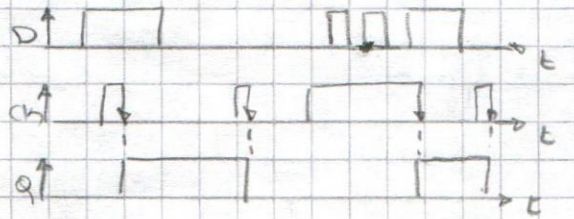
D	Q_{t+1}
0	0
1	1

Tabla de estados

Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

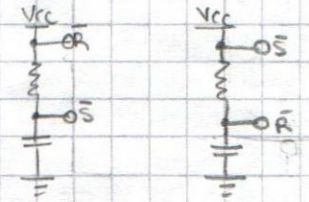
Diagrama de los estados





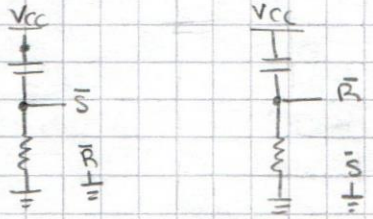
Condiciones iniciales

Activación con cero (o bajo)



CI=1 Q=1 | CI=0 Q=0

Activación con uno (o alto)



CI=1 Q=1 | CI=0 Q=0

NOTA

Ejercicio 2: VHDL: FF JK con reset sincrono y asincrono por flanco descendente

```
Entity FFJK is
   Port ( J,K,R,Ck : IN std_logic;
          Q : out std_logic; );
end FFJK

Architecture FF of FFJK is
begin
   Process (Ck, R)
   begin
      if (.....) then
         Q<='0';
      elsif( Ck'event and Ck='0')then
         if (J='0' and K='1')then
            Q<='0';
         elsif( J='1' and K='0') then
            Q<='1';
         elsif( J='1' and K='1') then
            Q<= (not Q)
         end if
      end if
   end process
end FF
```

síncrono: Ck'event and Ck='0' and R='1'

asíncrono: R='1'

Ck='0' flanco descendente
Ck='1' " positivo

VHDL: FFD con reset asincrono, act por flanco descendente

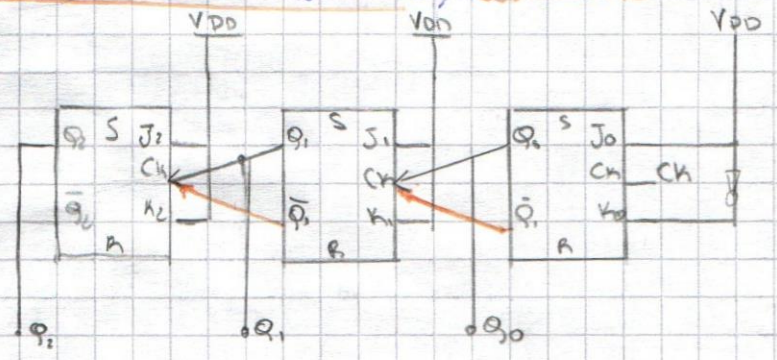
```
Entity FFD is
   port ( D,ck,R : in std_logic;
          Q;Qn: out std_logic; );
end FFD

Architecture FF of FFD is
begin
   Process (R,ck)
   begin
      if (R='1') then
         Q<='0'
      elsif (ck'event and clk='0' then)
         if ( D='0' then
            Q<='0';
         elsif( D='1') then
            Q<='1';
         end if
      end if
   end process
end FF
```

Asincrono

NOTA

Contadores (cuenta los pulsos de reloj y almacena el nº de cuenta)

- Contador asíncrono ascendente; descendente



Ejercicio - Contador síncrono con FF JK modulo 5 up/down con CI=000 m=5 estados

1) $2^n = 5 \rightarrow n = 3$ ff JK

2) Tabla de estado del ff JK

Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Diagrama de estado

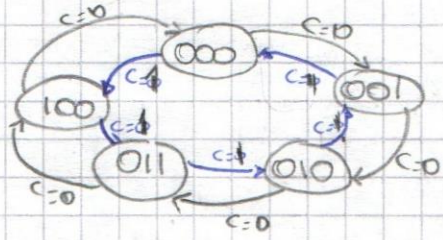


Tabla de estado el contador

	t			t+1								
C	Q_2	Q_1	Q_0	Q_2	Q_1	Q_0	J_2	K_2	J_1	K_1	J_0	K_0	Reset (fr)
0	0	0	0	0	0	1	0	x	0	x	1	x	0
0	0	0	1	0	1	0	0	x	1	x	x	1	0
0	0	1	0	0	1	1	0	x	x	0	1	x	0
0	0	1	1	1	0	0	1	x	x	1	x	1	0
0	1	0	0	0	0	0	x	1	0	x	0	x	0
0	1	0	1	x	x	x	x	x	x	x	x	x	1
0	1	1	0	x	x	x	x	x	x	x	x	x	1
0	1	1	1	x	x	x	x	x	x	x	x	x	1
1	0	0	0	1	0	0	1	x	0	x	0	x	0
1	0	0	1	0	0	0	0	x	0	x	x	1	0
1	0	1	0	0	0	1	0	x	x	1	1	x	0
1	0	1	1	0	1	0	0	x	x	0	x	1	0
1	1	0	0	0	1	1	x	1	1	x	1	x	0
1	1	0	1	x	x	x	x	x	x	x	x	x	1
1	1	1	0	x	x	x	x	x	x	x	x	x	1
1	1	1	1	x	x	x	x	x	x	x	x	x	1

NOTA

3) Determinar los valores de J y K de c/ FF

$C Q_2$ \ Q_1Q_0	00	01	11	10
00	0	0	1	0
01	X	X	X	X
11	X	X	X	X
10	1	0	0	0

$J_2 = C\bar{Q}_1\bar{Q}_0 + CQ_1Q_0$

$K_2 = 1$

$J_1 = \bar{C}Q_0 + CQ_2$

$K_1 = \bar{C}Q_0 + CQ_1\bar{Q}_0$

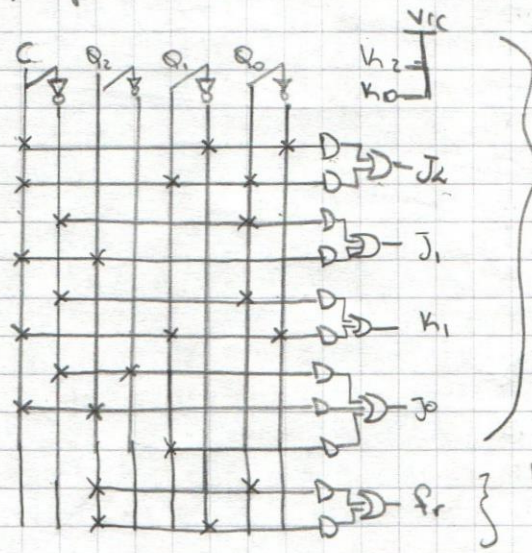
$J_0 = \bar{C}\bar{Q}_2 + CQ_2 + Q_1$

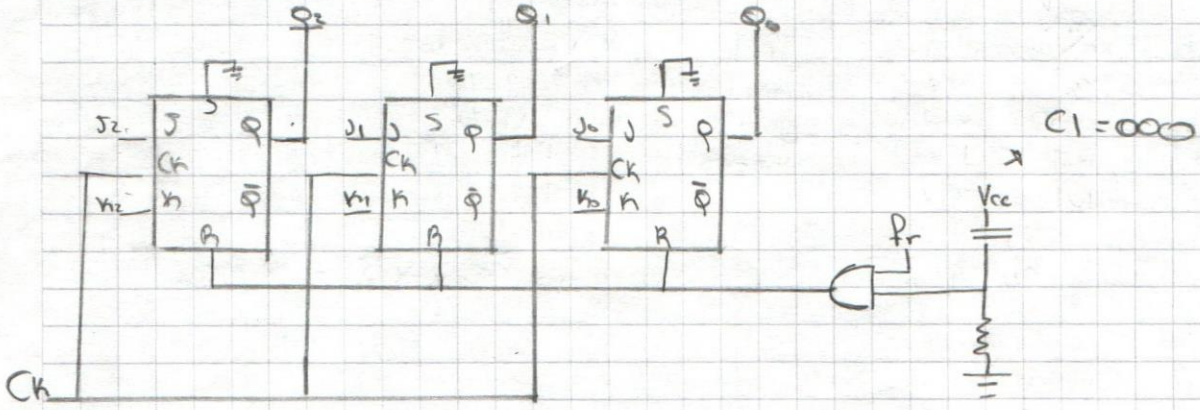
$K_0 = 1$

$C_2 Q_1$ \ Q_0	00	01	11	10
00				
01				
11				
10				

$f_r = Q_2Q_0 + Q_2Q_1$

4) Implementación





NOTA

Ejercicio

Contador FF-JK octal de n pares e impares según control X (X=0 par, X=1 impar)

	A	B	C
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Tabla FF-JK

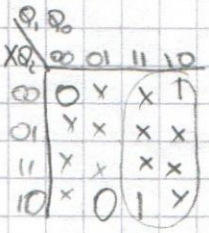
Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

$M=4$

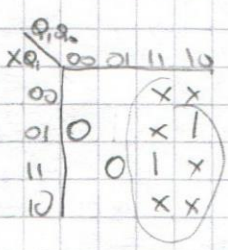
$n=3$ $\quad 2^n = 4$

$\therefore$ 3 FF JK

t			t+1									Reset
X	Q_2	Q_1	Q_0	Q_2	Q_1	Q_0	J_2 K_2	J_1 K_1	J_0 K_0	P_r
0	0	0	0	0	1	0	0 x	1 x	0 x	0
0	0	0	1	x	x	x	x x	x x	x x	1
0	0	1	0	1	0	0	1 x	x 1	0 x	0
0	0	1	1	x	x	x	x x	x x	x x	1
0	1	0	0	1	1	0	x 0	1 x	0 x	0
0	1	0	1	x	x	x	x x	x x	x x	1
0	1	1	0	0	0	0	x 1	x 1	0 x	0
0	1	1	1	x	x	x	x x	x x	x x	1
1	0	0	0	x	x	x	x x	x x	x x	1
1	0	0	1	0	1	1	0 x	1 x	x 0	0
1	0	1	0	x	x	x	x x	x x	x x	1
1	0	1	1	1	0	1	1 x	x 1	x 0	0
1	1	0	0	x	x	x	x x	x x	x x	1
1	1	0	1	1	1	1	x 0	1 x	x 0	0
1	1	1	0	x	x	x	x x	x x	x x	1
1	1	1	1	0	0	1	x 1	x 1	x 0	0

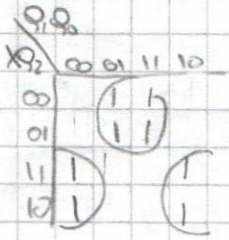


$J_2 = Q_1$



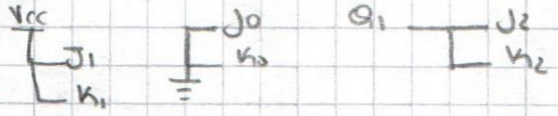
$K_2 = Q_1$

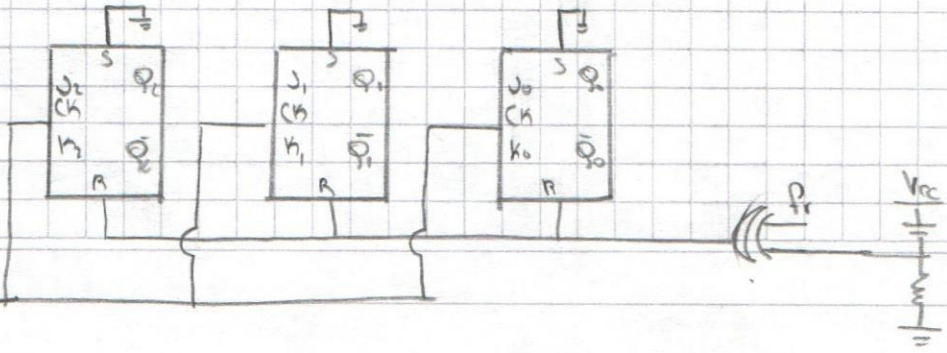
$J_1 = 1$
$K_1 = 1$
$J_0 = 0$
$K_0 = 0$



$P_r = \bar{X}Q_0 + X\bar{Q}_0 = X \oplus Q_0$

Implementación $C_1 = 000$





NOTA

Ejercicio 7: Contador de código gray de 4 bits usando FFD. Obtener una salida aux que presente el código BCD (C1=000)

Tabla de estado FFD

Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

$2^n = 16 \therefore n = 4$ FFD

t				t+1							
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0	D_3	D_2	D_1	D_0
0	0	0	0	0	0	0	1				
0	0	0	1	0	0	1	1				
0	0	1	0	0	0	1	0				
0	0	1	1	0	1	1	0				
0	1	0	0	0	1	1	1				
0	1	0	1	0	1	0	1				
0	1	1	0	0	1	0	0				
0	1	1	1	1	1	0	0				
1	0	0	0	1	1	0	1				
1	0	0	1	1	1	1	1				
1	0	1	0	1	1	1	0				
1	0	1	1	1	0	1	0				
1	1	0	0	1	0	1	1				
1	1	0	1	1	0	0	1				
1	1	1	0	1	0	0	0				
1	1	1	1	0	0	0	0				

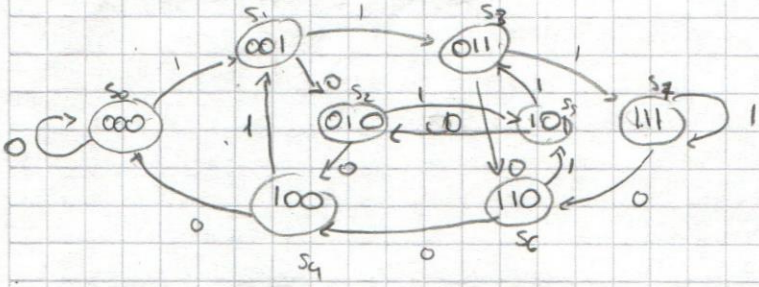
VHDL:

NOTA

Shift register

(8) Ejercicio 11: generar una secuencia binaria 01001011 usando registro de desplazamiento

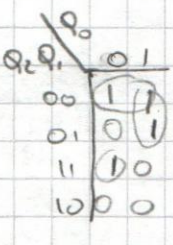
1) Diagrama de Brujn (8 estados)



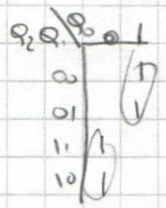
2) Tabla de estados

$Q_2 Q_1 Q_0$	Entrada $=J_0$	Secuencia
0 0 0	1	0
0 0 1	1	1
0 1 1	1	0
1 1 1	0	0
1 1 0	1	1
1 0 1	0	0
0 1 0	0	1
1 0 0	0	1

3) Karnaugh

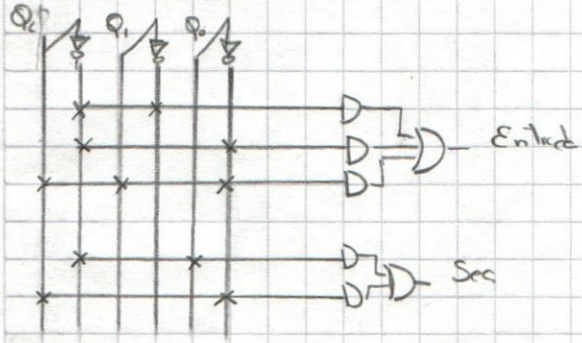


Ent: $\bar{Q}_2\bar{Q}_1 + \bar{Q}_2 Q_0 + Q_2 Q_1 \bar{Q}_0$

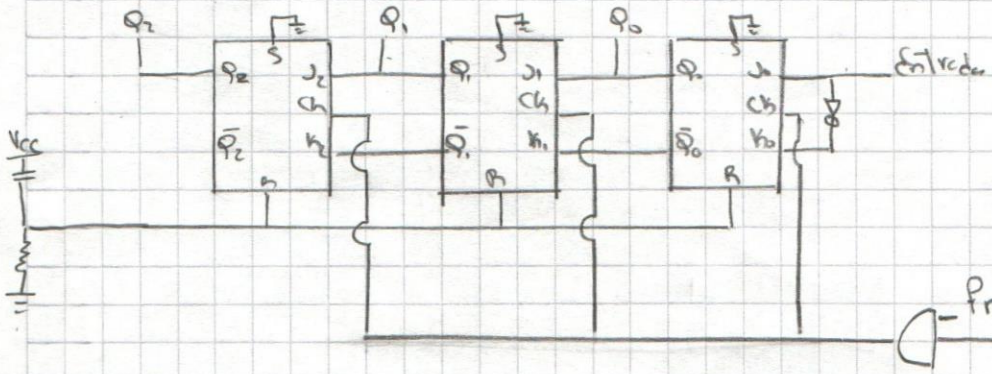


Sec: $\bar{Q}_2 Q_0 + Q_2 \bar{Q}_0$

4) Implementación



$C1: 000$



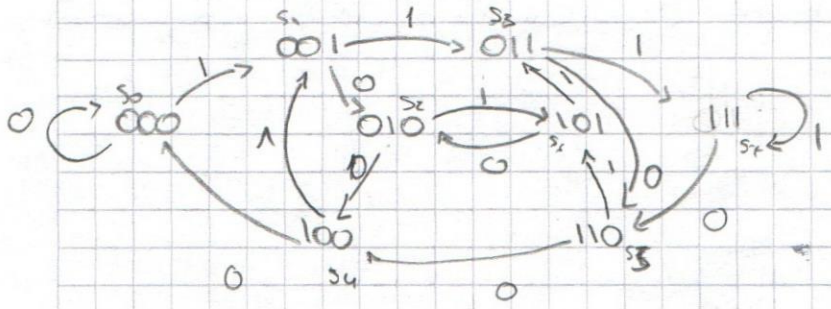
f_r = estados no usados

(hago con Karnaugh y lo que da lo pongo ahí)

NOTA

(a) Ejercicio 14: 2 secuencias (long 7; 5) m=1 genera secuencia 7
m=0 " " 5

Diseñar un ~~generador~~ de shift register utilizando ~~las~~ realimentación con XOR



Sec
m=0 $S_1, S_3, S_7, S_5, S_6, S_2, S_4$
m=5 S_1, S_3, S_7, S_5, S_4

m	Q_2	Q_1	Q_0	Ent.
1	0	0	1	1
1	0	1	1	1
1	1	1	1	0
1	1	1	0	1
1	1	0	1	0
1	0	1	0	0
1	1	0	0	0
0	0	0	1	1
0	0	1	1	1
0	1	1	1	0
0	1	1	0	0
0	1	0	0	1

Q_2Q_1 \ Q_0	0	1
00		(1)
01		(1)
11	(1)	(1)
10	(1)	

$Ent|_{m=1} = \bar{Q}_2 Q_0 + Q_2 \bar{Q}_0 = Q_2 \oplus Q_0$

Q_2Q_1 \ Q_0	0	1
00	(X)	(1)
01	X	(1)
11	0	0
10	(1)	(X)

$Ent|_{m=0} = \bar{Q}_1 \bar{Q}_0 + Q_2 Q_0$

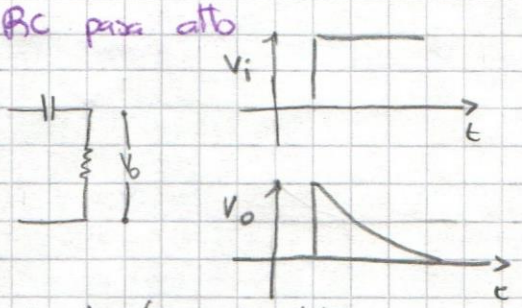
NOTA

Circuito de Tiempo

HOJA N° 1

FECHA

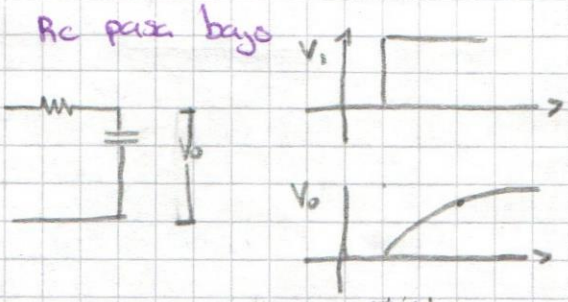
RC para alto



$$V_o = V_f + (V_i - V_f)e^{-t/\varepsilon}$$

$$t = RC \ln\left(\frac{V_f - V_i}{V_f - V_o}\right)$$

Rc para bajo



$$V_o = V_f(1 - e^{-t/\varepsilon})$$

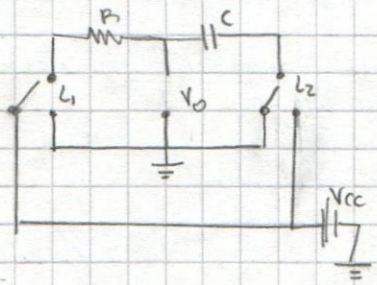
$$t = RC \ln\left(\frac{V_f}{V_f - V_o}\right)$$

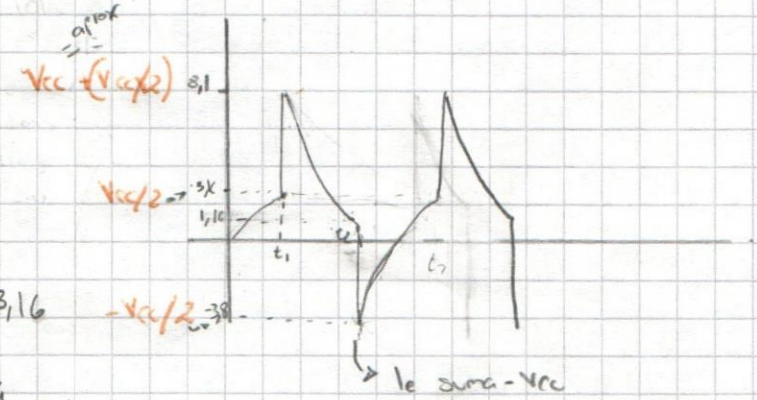
V_f: V_{max} final

V_o: V en el punto

Ejercicio para bajo / para alto

$R = 10k$ $C = 0{,}1\mu F$ $V_{cc} = 5V$ $t_1 = 1ms$ $\varepsilon = 1ms$ $t_2 = 2ms$


R
C
t_1
V_o
t_2
V_{cc}



P/bajo $V_o = V_{cc}(1 - e^{-t/\varepsilon}) = 5(1 - e^{-1}) = 3{,}16$

P/alto $V_o = \overset{=0}{V_f} + (V_i - V_f)e^{-t/\varepsilon} = 8{,}16\, e^{-2/1} = 1{,}1$

$V_{o3} = 3{,}1\,(1 - e^{-3}$

Osciladores

$$f = \frac{1}{2 \cdot n \cdot t_{pd}}$$

n = n° de inversores

t_{dp} = t° de propagación por inversor

$$Dcty = \frac{T_2}{T} = \frac{T_2}{T_1 + T_2}$$

NOTA

Ejer. 4: Osciladores con 3 inversores

NOTA

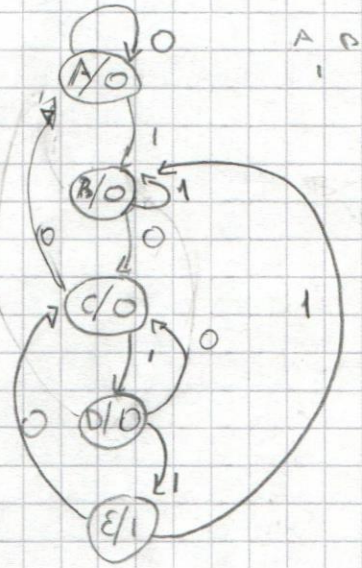
Circuitos secuenciales
(maq. de estado)

HOJA Nº 1

FECHA

A

1) Detecta secuencia 1011 Moore $C_1 = 000$



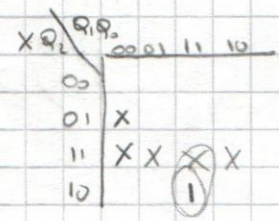
A B C D E
1 0 1 1

Tabla de estados:

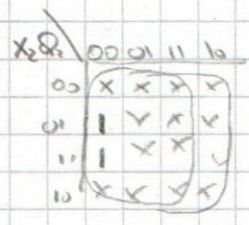
Q_t	Q_{t+1}	D	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

A	000
B	001
C	010
D	011
E	100
	101
	110
	111

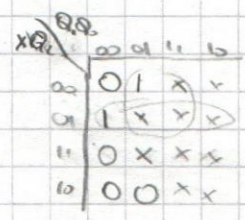
X	t	t+1	t $Q_2 Q_1 Q_0$	t+1 $Q_2 Q_1 Q_0$	J_2 K_2	J_1 K_1	J_0 K_0	Salida
0	A	A	000	000	0 x	0 x	0 x	0
0	B	C	001	010	0 x	1 x	x 1	0
0	C	A	010	000	0 x	x 1	0 x	0
0	D	C	011	010	0 x	x 0	x 1	0
0	E	C	100	010	x 1	1 x	0 x	0
1	A	B	000	001	0 x	0 x	1 x	0
1	B	B	001	001	0 x	0 x	x 0	0
1	C	D	010	011	0 x	x 0	1 x	0
1	D	E	011	100	1 x	x 1	x 1	0
1	E	B	100	001	x 1	0 x	1 x	1



$J_2 = X Q_1 Q_0$

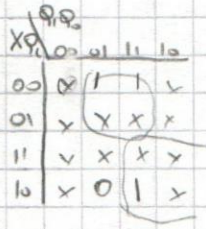


$K_2 = 1$



$J_1 = \bar{X} Q_2 + \bar{X} Q_0$

$K_1 = \bar{X} \bar{Q}_0 + X Q_0$

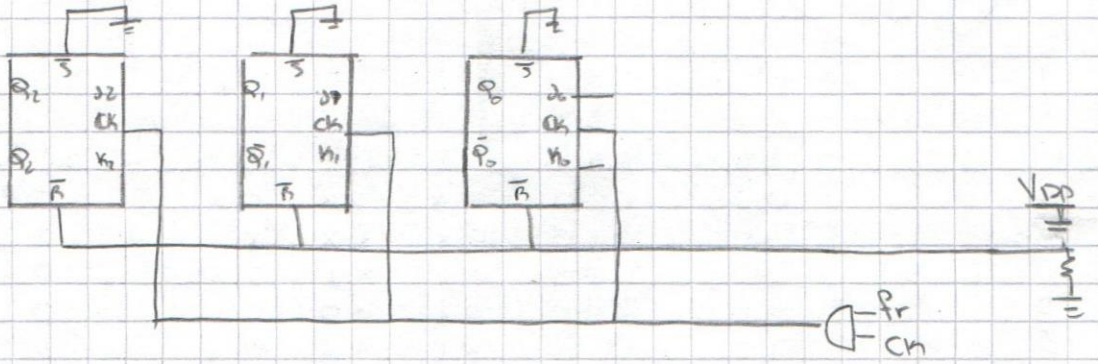


$K_0 = \bar{X} Q_0 + X Q_1$

$J_0 = X$

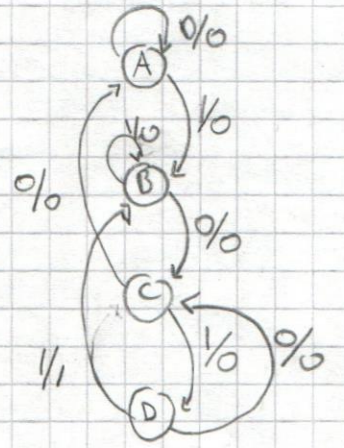
Salida: $\bar{X} Q_2 \bar{Q}_1 \bar{Q}_0 + X Q_2 \bar{Q}_1 \bar{Q}_0 = Q_2 \bar{Q}_1 \bar{Q}_0$

NOTA



NOTA

2) Detecta secuencia 1011 mealy C1: 00



A	00
B	01
C	10
D	11

tabla de estados

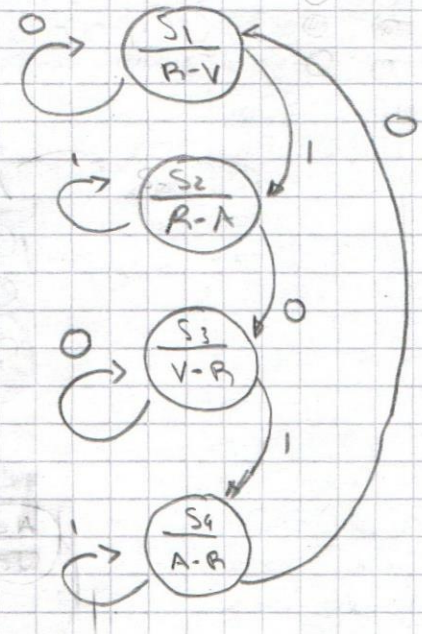
Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

				t	t+1		
X	t	t+1	Salida	Q_1 Q_0	Q_1 Q_0	J_1 K_1	J_0 K_0
0	A	A	0	0 0	0 0	0 x	0 x
0	B	C	0	0 1	1 0	1 x	x 1
0	C	A	0	1 0	0 0	x 1	0 x
0	D	C	0	1 1	1 0	x 0	x 1
1	A	B	0	0 0	0 1	0 x	1 x
1	B	B	0	0 1	0 1	0 x	x 0
1	C	D	0	1 0	1 1	x 0	1 x
1	D	B	1	1 1	0 1	x 1	x 0

NOTA

Ejercicio 23) Diseñe una M.E. Moore que implemente los semáforos cuando $x=1$ semáforos pasan a intermitencia

	Semáforo 1 R A V	Semáforo 2 R A V
R-V =	1 0 0	0 0 1



S_1	0 0
S_2	0 1
S_3	1 0
S_4	1 1

Tabla de estados

t	t+1	D
0	0	0
0	1	1
1	0	0
1	1	1

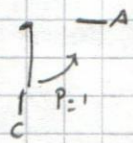
X	t	t+1	Salida	t	t+1	D_1 D_0
0	S_1	S_1	100 001	00	00	0 0
0	S_2	S_3	100 010	01	10	1 0
0	S_3	S_3	001 100	10	10	1 0
0	S_4	S_1	010 100	11	00	0 0
1	S_1	S_2	100 001	00	01	0 1
1	S_2	S_2	100 010	01	01	0 1
1	S_3	S_4	001 100	10	11	1 1
1	S_4	S_4	010 100	11	11	1 1

↳ la salida está en M
va en la prueba

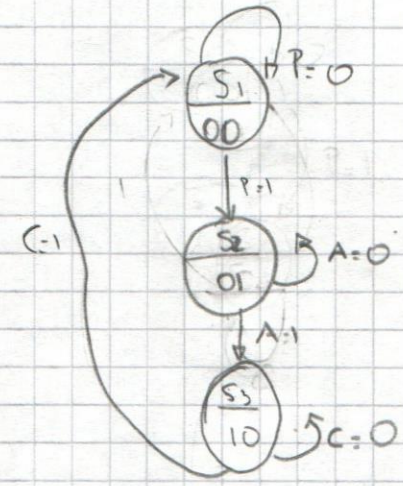
NOTA

Guía de autómatas

1)



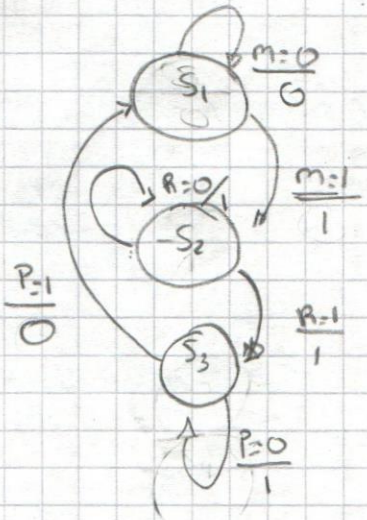
MOORE



Cerrada → S_1 : 00
abriendo → S_2 : 01
abierto → S_3 : 10

A: tope de apertura
C: Tope de cierre
P: pulsador

2) M → 1
P → 0



Asignación de estados

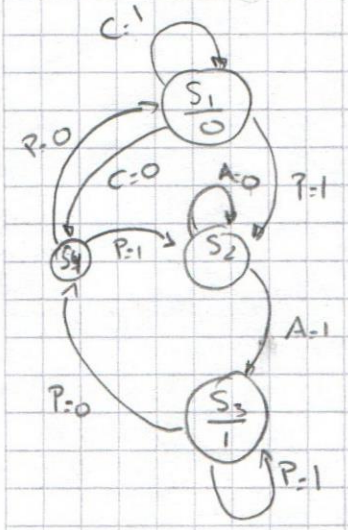
S_1 = Detenido 00
S_2 = marcha 01
S_3 = reposo 10

		t	t+1	Salida
M	R P	$Q_1 Q_0$	$Q_1 Q$	
0	X X	00	00	0
1	X X	00	01	1
X	0 X	01	01	1
X	1 X	01	10	1
X	X 0	10	10	1
X	X 1	10	00	0

NOTA

3) P = detec. de personas
A = " Puerta abierta
C = " " cerrada

S_1 = Cerrad
S_2 = abriendo / cerrando → intermedio
S_3 = abierto
S_4 cerrnd

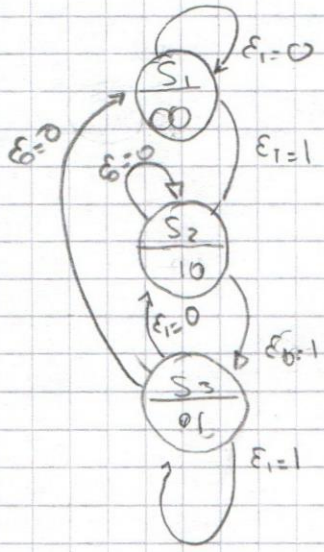


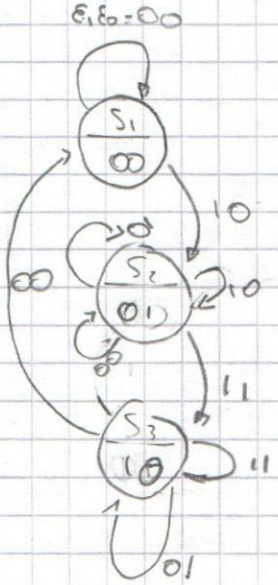
Pxx →

ε_1 ε_0	A B	
0 0	0 0	
0 1	0 1	largo
1 0	1 0	corto
1 1		

ε_1
ε_0 : poner salida

4) largo B = 1
corto A = 1





NOTA

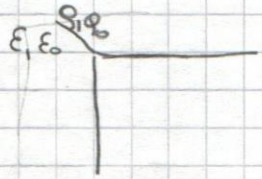
00 S_0
01 S_2
10 S_3
11

t	t_{+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

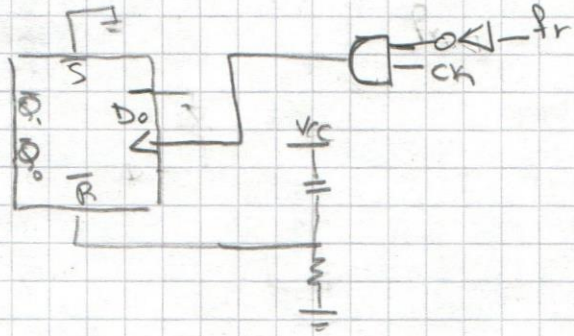
HOJA N° 4

FECHA

E_1 E_0	S_t	S_{t+1}	S_2 S_0	Q(t)	Q(t+1)	D_1 D_0	Reset
0 0	S_1	S_1	0 0	0 0	0 0	0 0	
0 0	S_2	S_2	0 1	0 1	0 1	0 1	
0 0	S_3	S_1	0 1	1 0	0 0	0 0	
				1 1			1
0 1	S_1	X	X X	0 0	X X	X X	1
0 1	S_2	S_2	0 1	0 1	0 1	0 1	
0 1	S_3	S_3	1 0	1 0	1 0	1 0	
				1 1			1
1 0	S_1	S_2	0 0	0 0	0 1	0 1	
1 0	S_2	S_2	0 1	0 1	0 1	0 1	
1 0	S_3	X	X X	1 0	X X	X X	1
				1 1			1
1 1	S_1	X	X X	0 0	X X	X X	1
1 1	S_2	S_3	0 1	0 1	1 0	1 0	
1 1	S_3	S_3	1 1	1 0	1 0	1 0	
				1 1			1



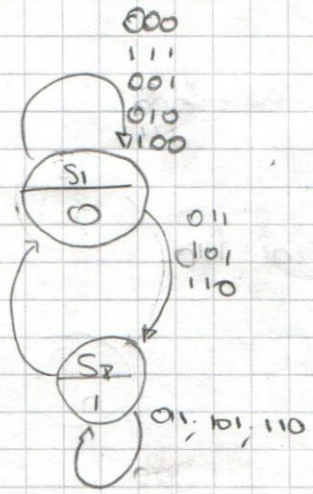
D_1
D_0
S_1
S_0
f_{ext}

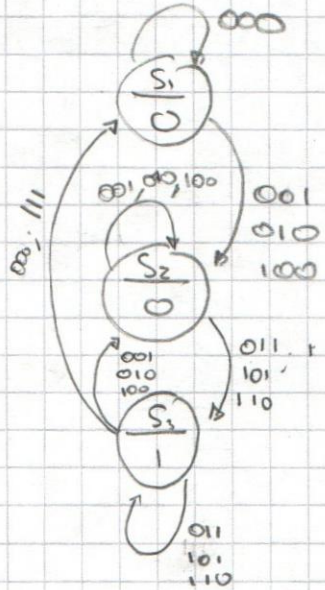


$S_1 = 0$
$S_2 = 1$

5)

P_3	P_2	P_1	t	t_{+1}	$Sal = D_0$
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	0	1	0
1	1	0	0	1	0
1	1	1	0	0	0
0	0	0	1	0	1
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	1	0	1
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	0	1





NOTA

Sal

00	Parado
01	izq
10	derec

Estado

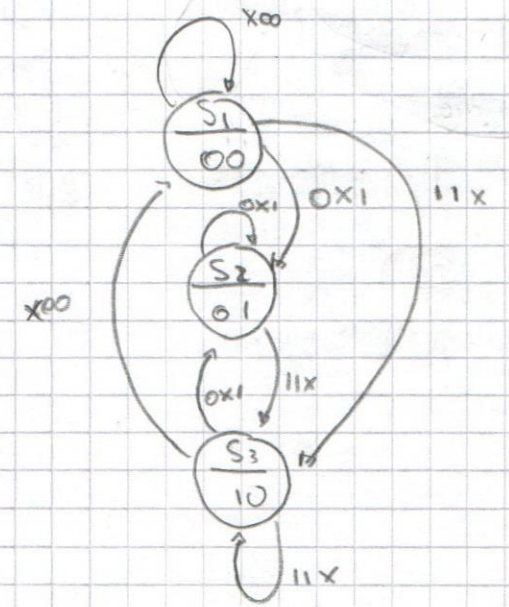
00	S_1
01	S_2
10	S_3
11	

⑥ D = 1 derecha l = 1

I = 1 izq

l	D	I	Mov	t	t+1	Sal.
0	0	0	00	S_1	S_1	00
0	0	1	01	S_1	S_2	
0	1	0	10	S_1	S_3	
0	1	1	01	S_1	S_2	
1	0	0	00	S_1	S_1	
1	0	1	01	S_1	S_2	
1	1	0	10	S_1	S_3	
1	1	1	10	S_1	S_3	

l	D	I	t	t+1	Sal
x	0	0	S_1	S_1	00
0	x	1	S_1	S_2	00
1	1	x	S_1	S_3	00



NOTA

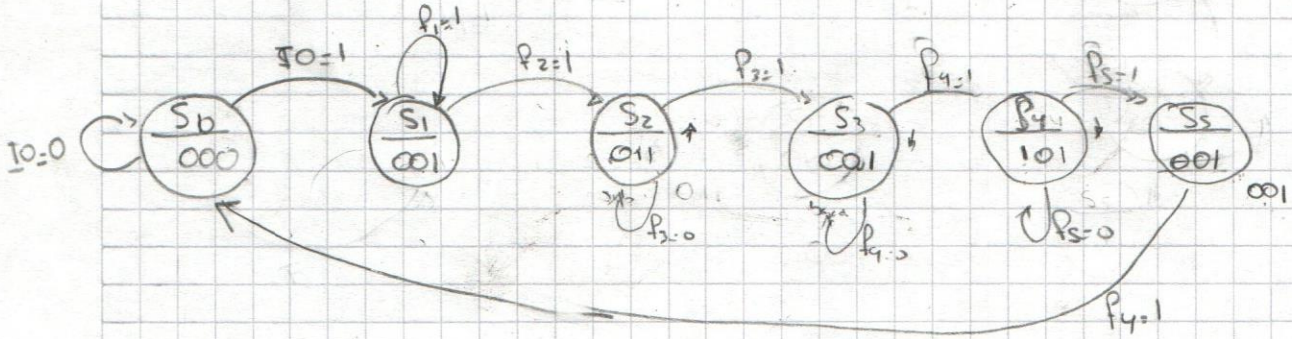
$S_0 = 000$
$S_1 = 001$
$S_2 = 010$
$S_3 = 011$
$S_4 = 100$
$S_5 = 101$

HOJA N° 5

FECHA

1) Ejercicio de la prensa de inyección de plásticos

	A											
	I0	P_2	P_3	P_4	P_5	t	t+1	sal	D_2	D_1	D_0
	0	x	x	x	x	000	000	000			
	1	x	x	x	x	000	001	000			
	x	0	x	x	x	001	001	001			
	x	1	x	x	x	001	010	001			
	x	x	0	x	x	010	010	011			
	x	x	1	x	x	010	011	011			
	x	x	x	0	x	011	011	001			
	x	x	x	1	x	011	100	001			
	x	x	x	x	0	100	100	101			
	x	x	x	x	1	100	101	101			
	x	x	x	0	x	101	101	001			
	x	x	x	1	x	101	000	001			



NOTA

NOTA